Allocating What Programs Touch: Non-Binding Per-Site Reservations from Measured Memory Footprints

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

A program asks the allocator for the memory it might need, and the operating system and a cluster scheduler must honour every request, although the program often touches only part of it. We use MemPrint, a Pin-based footprint profiler, to measure what each allocation site requests and what the program touches in each block, and we pass these measurements to the compiler and the runtime as *non-binding reservations*. The system has three parts. First, a per-site mode of the Pin tool records each large block's touched bytes, highest touched offset and touched page ranges, the calling context that allocated it, and the growth of every container; zeros written over untouched memory do not count as use. Second, `MemPrintResize`, an LLVM 21 pass plugin, routes every allocation through a runtime with a site identifier that survives edits elsewhere in the program, raises the capacity of growing `std::vector`s, strings and hash tables per call site, with element sizes from the debug information, and can embed the reservations in the binary. Third, `libmpr`, linked or preloaded, commits only the predicted prefix or ranges of a block and maps the rest `MAP_NORESERVE`, so that an under-prediction costs commit accounting and never correctness; it also skips the zeroing of pages a program has never touched. Every program we ran produced its original output. On llama.cpp, which sizes and zeroes a KV cache for an 8192-token context, `libmpr` reduces peak commit by 86% and peak RSS by 54% with a 135M-parameter model (90% and 45% with a 360M model), and profiles from other model sizes do as well. It reduces commit by 85% on llama2.c, whose KV caches are used in one range per layer, by 47% on darknet AlexNet inference, and by 22% on miniVite with the pass's per-call-site container capacities. Across a 23-program HPC and interpreter corpus, where most programs already hold about what they touch, it changes no output and no total RSS, keeps run time within −5% to +7%, and reduces total peak commit by 10.7%. The pass adds 0 to 3% to compile time, and 96.6% of its site identifiers survive an edit that changed 19% of the previous ones. On 21 LLM inputs that no profile saw, the savings hold with one profile per model, and a *periodic* reservation that combines the profile's layout with the run's token count leaves no KV cache used beyond its commitment, where a reservation from the profile alone left up to 27 MB. Finally, MemPrint's static interpreter now produces the same per-site profile from source: on GAP's kernels it finds every site Pin finds, with total high water within 0.5%, and its reservations perform as Pin's do. `libmpr` can also take the profile itself, from the pages each block touched, in one native run: over 28 configurations its reservations commit what the Pin profile's do (2,391 against 2,383 MB) after 95 s of profiling instead of 21,676 s. Mapping threads' stacks `MAP_NORESERVE` cuts a further 60% of the commit of miniVite and llama.cpp, and a second run parameter, the prompt's length, leaves nothing beyond the commitment on 9 of 10 unseen llama.cpp inputs. Against three published memory-provisioning policies, Pond, dynamic provisioning and KS+, which change what a system holds for a job but not what the job commits or touches, `libmpr` holds 23% less memory over the 29 workloads, and in a simulated

cluster it is the only one that raises throughput: 25.1 jobs per hour against 21.6 for the request and at most 21.5 for the three, with every policy's own costs counted. Its native-speed profile costs 1% of the cluster's time; a Pin profile taken in the same stream would cost 88% of its throughput, so Pin profiles must be taken offline. With dynamic provisioning on top, it gives the shortest response time of any policy and needs 5.0 instead of 6.0 TB for the same service.

1 Introduction

A program's memory request and its memory use are different quantities, and every layer below the program plans for the request. The allocator maps what is requested, the kernel charges the mapping to the commit limit [22], and a batch scheduler on a cluster with disaggregated memory reserves the request, or an estimate of it, before it places the job [47, 20]. Our earlier work built MemPrint, a Pin-based profiler [27, 33] that measures the bytes a program touches over its run, and showed that a scheduler which knows a job's footprint curve rather than its request can pack jobs much more tightly [32]. However, a scheduler that reserves less than a program requests does not change what the program asks for. The program still allocates the full request, and the kernel still charges it.

The gap between request and use is large in some programs. darknet [39] running AlexNet inference commits 534.7 MB and touches 270 MB, because it allocates training-only weight-update buffers that inference never uses. llama.cpp [12] sizes its KV cache for the model's full 8192-token context and then writes zeros over all of it, so a 32-token completion holds 189 MB of cache in RAM and uses 0.4% of it. miniVite's edge list grows by doubling, so for a moment the old and new buffers are both live [13].

Therefore, we ask whether a measured footprint can be passed down to the allocator, so that the program obtains what it touches rather than what it requests, without trusting the measurement for correctness. Specifically, we treat MemPrint's per-site measurements as *non-binding reservations.* A block always receives every byte the program requested, but only the predicted part is committed, and the rest is mapped so that the kernel backs it only if the program touches it. A wrong prediction then costs accounting and never causes a fault the program can observe. *To the best of our knowledge, this is the first compiler pass and runtime that commit only the profiled part of each heap block, leaving the rest to be backed on demand.* Existing profile-guided allocation work, such as LLVM's MemProf, changes where an object is placed but commits every requested byte [26], and existing container-sizing tools give advice to the programmer or replace data structures in managed runtimes [11, 29] (Section 3).

In this work, we make the following contributions:

- We extend MemPrint's Pin tool to profile requested against touched bytes per allocation site, with each block's touched bytes, highest touched offset and touched pages measured exactly when the block is freed (Section 5).

- We extend the profile with calling contexts, so that a site reached through an allocation wrapper is split by its callers, with touched page ranges for blocks used sparsely across their length, with container growth chains, and with zeroing of untouched memory left out of use (Section 5, Section 6).

- We present `MemPrintResize`, an LLVM pass plugin that gives every allocation call a site identifier that survives edits elsewhere in the program, routes address-taken allocators through thunks and large zeroing through the runtime, and raises the capacity of growing vectors, strings and hash tables per call site, with element sizes from the debug information (Section 7).

- We present `libmpr`, a runtime that commits a block's predicted prefix or ranges and maps the rest `MAP_NORESERVE`, matches reservations by calling context, grows blocks by moving page tables, skips the zeroing of pages that were never touched, and refuses address-keyed reservations profiled on another build (Section 8).
- We fit reservations as functions of a run parameter when a block's use depends on the run rather than on its size: a `periodic` model takes the layout from the profile and the token count from the run (Section 6).
- We extend MemPrint's static interpreter to produce the per-site profile from source, keyed by the pass's identifiers through its site map (Section 9).
- We evaluate the system on darknet, miniVite, two LLM inference engines and a 23-program corpus. It cuts llama.cpp's commit by 86% and its resident memory by 54%, and llama2.c's commit by 85%, with identical outputs and no time cost, and we measure what each refinement adds (Section 10). The savings hold on 21 unseen LLM inputs, and the parameterised reservation follows their lengths (Section 10.7); the static profiles of GAP agree with Pin's site by site (Section 10.11).
- We compare `libmpr` with three published memory-provisioning policies, Pond, dynamic provisioning and KS+, job by job and in a simulated cluster, with each policy's own costs (profiling, resizes, migrations, retries): `libmpr` is the only one that raises throughput, because it shrinks the jobs rather than how they are held, and it composes with dynamic provisioning (Section 10.17, Section 10.18, Section 10.19).
- We report what the measurements show about footprint estimation: where miniVite's extra memory comes from, why the bytes a program touches can exceed its resident set, and why the time axis of a footprint curve matters more than its unit (Section 11).

2 Background

2.1 Three measures of a program's memory

We use three measures throughout, and the results depend on which one is meant.

Bytes touched. MemPrint's footprint is the number of distinct bytes the program has read or written in memory that is still live, measured from every memory reference under Pin [33]. With `-track_frees`, a freed range leaves the footprint. Bytes touched are a property of the program and its input, and they do not depend on the allocator or on the page size.

Resident set (RSS). The kernel counts the pages of a process that are in RAM. RSS includes file-backed pages, such as a memory-mapped model file and the code of every shared library, and it counts a transparent huge page as 2 MB [23]. We sample it from `/proc/<pid>/smaps_rollup` every 20 ms and report the peak sample. RSS is the quantity the kernel reclaims and the out-of-memory killer acts on, and it is the "RSS truth" of the scheduling study [32].

Commit. Linux charges private writable mappings to the commit limit when they are created [22]. We measure commit as the total size of the mappings that `/proc/<pid>/smaps` flags `ac` (accountable), sampled with RSS. A mapping created with `MAP_NORESERVE` is not charged. Under the default heuristic overcommit policy commit is only advisory, but under strict overcommit (`vm.overcommit_memory=2`) it is a hard limit, and in both cases it is the memory the system has promised to the process.

System	Profile	Action	Automatic	Wrong profile
LLVM MemProf [26]	access density and lifetime per allocation context	hot or cold hint to `operator new` (placement)	yes (compiler + TCMalloc)	placement only
libstdc++ profile mode, Perflint [24, 11]	container operations per call stack	advice: initial size, container choice	no (programmer)	–
Chameleon, Brainy [42, 15]	collection usage	select a collection implementation	partly	slower or larger collection
Makor et al. [29]	data-structure use per allocation site	replace with a smaller implementation (GraalVM Native Image)	yes	guarded by the implementation
Odaira (IBM patent) [34]	highest index accessed per allocation context	JIT sets the initial array size	yes (Java JIT)	array grows by copying
Xu et al., DHAT [44, 45, 43]	cost and benefit, or bytes accessed, per site	report bloat	no (programmer)	–
This work	bytes touched, highest offset and touched page ranges per block, site and calling context; container growth chains per call site	commit the predicted prefix or ranges, the rest `MAP_NORESERVE`; raise container capacities per call site; skip zeroing untouched pages	yes (pass or preload)	costs commit accounting only

Table 1: Profile-guided work on heap allocations, and how a wrong profile affects each.

2.2 Why untouched memory is often free already

Linux backs anonymous memory lazily. A large `malloc` or `calloc` block is served by `mmap`, and a page of it becomes resident only when the program first writes it, or reads it, in which case the kernel maps a shared zero page that RSS does not count. Therefore, a block that the program never touches costs commit but no RSS. darknet's 250 MB of unused weight-update buffers are an example. Reducing RSS requires a change in how memory is obtained, not a smaller reservation. In our measurements RSS fell only where the program, its allocator, or the kernel touched pages the program did not need: vector growth that keeps two buffers live, a zeroing `memset` over fresh memory, and transparent huge pages that back 2 MB on the first touch.

3 Related work

We searched the compiler, allocator, operating-system and LLM-serving literature for an existing pass that resizes heap allocations from a profile of what a program touches. To the best of our knowledge, no prior compiler pass commits only the profiled part of a heap block and leaves the rest to be backed on demand. The closest work falls into five groups, which Table 1 summarises.

Profile-guided compilation of allocation sites. LLVM's MemProf is the closest existing compiler work [26]. It records, per allocation context (a call stack of function, line and column), the allocation count, access counts at 64-byte granularity and lifetimes. Its `MemProfUse` pass matches the profile to allocation calls through debug information, clones functions under LTO to separate contexts, and rewrites `operator new` into a variant that carries a hot or cold hint, which an allocator such as TCMalloc uses to place the object. `MemPrintResize` shares this structure: an

instrumented profile, sites matched through the inlined debug location chain, and allocation calls rewritten to a runtime that accepts site information. However, MemProf changes only where an object is placed and leaves every requested byte committed, whereas `MemPrintResize` and `libmpr` change how much of each block is committed and how it is backed, and a site's reservation is a size model rather than a temperature. MemProf's profile is collected by a sanitizer-style runtime in the program itself, while ours comes from MemPrint's Pin tool, which needs no recompilation for the profiling run and measures bytes rather than 64-byte granules.

Profile-guided container sizing and selection. Perflint traces C++ standard library operations and reports where a better use exists [24], and libstdc++'s profile mode, its descendant, prints advice such as a vector's suggested initial size or a vector that is never filled to its reserved size [11]. The advice is for the programmer to apply, and libstdc++ notes that it depends on the profiled input. Chameleon selects Java collection implementations from runtime metrics [42], Brainy selects C++ data structures with machine-learned cost models [15], and Makor et al. automatically replace data structures at allocation sites in GraalVM Native Image with versions specialised to their profiled use, which reduces memory by up to 13.85% (2.94% on average for microservices) [29]. The closest to our vector capacity is an IBM patent by Odaira, in which a Java JIT records the highest index accessed per allocation context and sets the initial array size from it when it inlines the allocation site [34]. Our capacity reservation differs in that it can only raise the capacity a growing vector would have chosen, so a wrong prediction cannot shrink a block below its request, and it applies to C++ compiled ahead of time, with the profile taken from an unmodified binary.

Bloat detection and heap profiling. Xu et al. find Java data structures whose cost exceeds their benefit [44], and Xu and Rountev find containers that are overpopulated or underused [45]. Heap profilers that report lag, drag and void, the time a block is allocated but not yet or no longer used [40], and DHAT, which reports per allocation stack how many bytes were read and written and flags blocks that are not completely filled in [43], also diagnose over-allocation. All of them leave the fix to the programmer. MemPrint's `-sites` profile reports similar quantities, but the system acts on them without a source change.

Placement, compaction and huge pages. Mesh compacts the heap of unmodified C and C++ programs by merging pages whose live objects do not overlap [37], Maas et al. predict object lifetimes from allocation call stacks to place long-lived objects together [28], and HALO groups heap objects by profiled access affinity after link time [41]. These systems use per-site profiles to change placement, whereas we use them to change commitment and backing. jemalloc and TCMalloc's hugepage-aware Temeraire return freed memory to the kernel more eagerly than glibc [8, 14], but they still commit every requested byte. Ingens and HawkEye showed that transparent huge pages inflate the resident set of sparsely used memory, and they promote a region to a huge page only when enough of it is used or deduplicate zero-filled huge pages in the kernel [18, 36]. `libmpr`'s huge-page policy addresses the same bloat from user space, for the specific pages whose zeroing it skipped. Yang et al. measured the cost of zeroing new memory in managed runtimes and made zeroing cheaper with non-temporal and concurrent zeroing [46]. `libmpr` instead avoids writing zeros to pages that are already zero, which saves resident memory as well as time.

Calling contexts, stale profiles and partial commitment. Several ideas we build on come from profiling and operating systems. Calling-context profiling attributes a measurement to the chain of calls that led to it, recorded as a calling context tree [2] or encoded cheaply as a probabilistic calling context value per thread [5], and MemProf disambiguates allocation contexts by cloning functions under LTO [26]. We split a site by its caller frames, taken with the call frame information at allocation time, which is enough to separate the callers of an allocation wrapper. Sample-based profile-guided optimisation keys its profile by the line offset from a function's first line so that a profile outlives edits elsewhere in a file [6, 25], and stale profile matching recovers

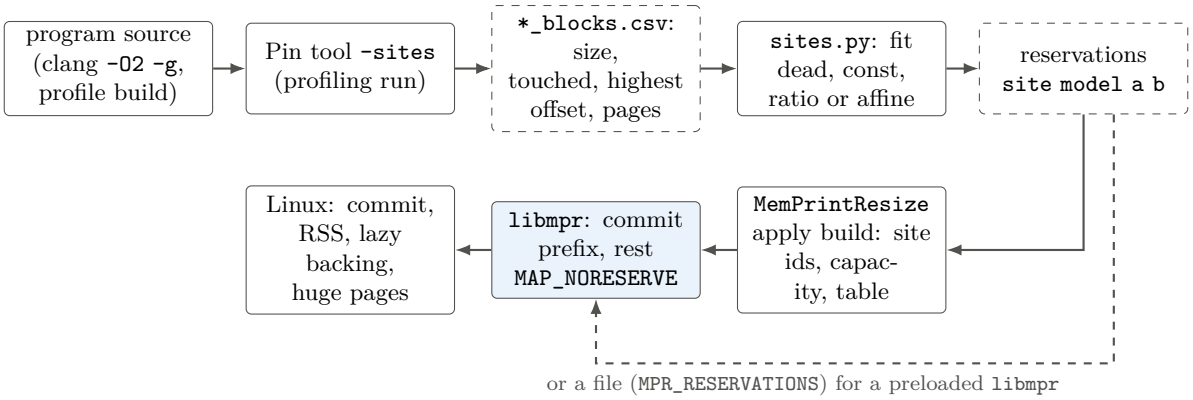


Figure 1: The pipeline. A profiling run of the program under MemPrint's Pin tool measures each large block; `sites.py` fits one reservation per allocation site; the reservations reach `libmpr` either embedded in a binary built with `MemPrintResize` or from a file when `libmpr` is preloaded into an unmodified binary.

a profile for a changed binary [3]. We use the offset scheme for site identifiers, and GNU build identifiers [10] to refuse address-keyed reservations on another build. Windows lets a program reserve address space and commit chosen pages of it later [30], and Linux's `MAP_NORESERVE` leaves a mapping out of the commit charge [22]. Our shaped reservations commit chosen ranges of a block from a profile, without a change to the program.

LLM serving and memory pools. PagedAttention allocates an LLM server's KV cache in fixed-size blocks as tokens arrive [17], which requires paged attention kernels and a memory manager in the serving framework. vAttention keeps the KV cache contiguous in virtual memory and backs it with physical GPU memory on demand through the CUDA virtual memory API, which removes the kernel changes but still requires changes to the serving framework [38]. Our llama.cpp result obtains the same kind of on-demand backing on the CPU for an unmodified engine: the cache keeps its full virtual size, and only the pages it writes are backed, because `libmpr` skips the zeroing that would otherwise touch the whole cache. At cluster scale, Pond predicts how much of a virtual machine's memory will stay untouched and places that part on a CXL pool [20], Zacarias et al. show that a 60% overestimate of memory demand reduces the throughput of a disaggregated system by about 25% [47] and resize each job's allocation to its use every five minutes [48], and KS+ predicts a workflow task's memory over time from earlier executions with other inputs [4]. All three change what a system holds for a job, not what the job commits or touches; Section 10.17 compares them with this work. These works motivate knowing what a job touches, which our earlier study measured for scheduling [32] and this work passes to the allocator.

4 Overview

Figure 1 shows the pipeline. First, the program runs once under MemPrint's Pin tool with `-sites`, which writes one row per large block. Second, `sites.py` fits each site's reservation from one or more such runs. Third, the reservations reach `libmpr`. A program can use the system in two ways. In the *preloaded* configuration, `libmpr` is loaded with `LD_PRELOAD` into an unmodified binary, a site is the return address of the allocation call, and the reservations come from a file. In the *pass* configuration, the program is built with `MemPrintResize`, a site is a hash the compiler computes, and the reservations can be embedded in the binary. Only the pass configuration can change a vector's growth capacity, because the capacity is computed inside inlined library code, before any

allocation call.

5 Per-site profiling in MemPrint

The `-sites` mode of the Pin tool runs with the exact splitter, `-track_frees` and the bytes-touched footprint, which keeps one bit per byte for every touched page. It records, for every large block, what the program asked for and what it did with it, and it writes four files: `*_blocks.csv` (one row per block of at least 64 KB), `*_sites.csv` (totals per site), `*_capacity.csv` (container growth, Section 5.4) and `*_images.csv` (the images that site keys name, with their paths).

5.1 Sites and calling contexts

The tool hooks the allocator's entry points (`malloc`, `calloc`, `realloc`, the aligned variants and every `operator new`). For the outermost allocator call of a thread it records the call's return address, written as the image name plus the address in the image file (for example `llama-completion+0x92266f`), which stays the same across runs despite address-space layout randomisation. Because `operator new` is hooked as an allocator, an allocation made by inlined standard-library code resolves to the program's own code. In a program built with `MemPrintResize`, the tool also hooks `__mpr_alloc` and the other runtime entry points, reads the site identifier from the argument the pass added, and names the site `id:0x...`.

A site is a single call, and a call inside an allocation wrapper serves every caller of the wrapper. llama.cpp allocates its KV cache, its compute buffers and its weights' scratch space through one function, `ggml_aligned_malloc`, so all of them share one site, although the program uses them very differently. Calling-context profiling distinguishes the callers of a function by the chain of calls that reached it [2, 5], and LLVM's MemProf records allocation contexts as call stacks for the same reason [26]. With `-site_context` k (2 by default), the tool records, for every allocation of at least 64 KB, the k return addresses above the allocation call. It takes them with `PIN_Backtrace`, which unwinds through the call frame information, so the program needs no frame pointers, and `libmpr` takes the same frames with glibc's `backtrace()` at run time. The frames are written with the block as image keys, `frame1|frame2`. Backtraces are taken only for large requests, so a program's many small allocations add no unwinding cost.

5.2 Block summaries

When a release (a `free`, a `realloc` that moves, or an `munmap`) is applied, the tool is about to clear the block's bits from the footprint. Just before it does, it reads the block's touched bytes, the offset of its highest touched byte and its number of touched pages from the per-page bitmaps, and it writes the block's touched pages as runs, `first:count` relative to the block's first page, merging the closest runs when there are more than 64. Releases are queued in order with each thread's buffered references, so a summary sees every reference the program made to the block before freeing it. Blocks still live at exit are summarised at exit. `calloc`'s own zeroing of a reused heap chunk is measured by a probe queued at the `calloc` exit and recorded separately as `alloc_touched`.

5.3 Zeroing that does not count as a touch

A program that clears a buffer it has just allocated writes every byte of it, so a bytes-touched profile counts the whole buffer as used. llama.cpp clears its 189 MB KV cache this way and then writes a few rows of it, so the first profile reserved all of it. Writing zeros over memory that the program has

not yet touched does not change what the memory holds, and libmpr skips such writes (Section 8.5). With `-skip_zeroing`, on by default with `-sites`, the tool leaves these writes out of the footprint. It hooks the entry of every `memset` and `bzero` implementation in the C library (skipping the IFUNC resolvers that select among them, which the loader calls once), libmpr's `memset` and the pass's `__mpr_zero`. At each entry it queues, in order with the thread's buffered references, the range that the call clears: $[p, p + n)$ when the value is zero and $n \geq 4\,\text{KB}$, and an empty range otherwise. Stores made by the code of those routines are tagged when they are instrumented, and a tagged store into the current range counts as time but does not set the footprint's bits. A later read or write by the program touches the bytes as usual. A byte that was already touched stays touched, so the option changes only what zeroing of untouched memory contributes. Yang et al. measured that zeroing new memory costs managed runtimes 2.7 to 4.5% of their time on average and made it cheaper [46]. Here the point is that zeroing of untouched memory is not use.

5.4 Container growth chains

In a program built with `MemPrintResize`, a container that grows calls `__mpr_capacity` before it allocates its new buffer (Section 7.3). The tool hooks that call and `__mpr_growth`, and it attaches the capacity key, the call site that grew the container, to the buffer that the next allocation returns. Buffers with the same key form a growth chain when each new buffer is larger than the previous one and is allocated while the previous one is still live, which is how `std::vector`, `std::string` and hash tables grow. For each key, `*_capacity.csv` records the number of chains, the number of chains whose final buffer stayed below 64 KB and the largest of those, and, for the other chains, the final buffer's size and its highest touched offset, which is the bytes the container used.

5.5 Images and builds

An image key is an address in one build of an image and names unrelated code in another build. The tool writes the full path of every image it saw, and `sites.py` records each image's GNU build identifier, a hash that the linker writes into a note section [10], with the reservations (Section 8.6).

Tests. The suite checks exact values on programs written for the purpose: a dead block, a block touched only in a prefix, a block touched only at its tail, a `calloc` on a reused chunk, a `realloc` growth chain, a growing vector, an allocation wrapper called from two functions that use their blocks differently, and a 1 MB buffer cleared with `memset` and then written in its first 64 KB (touched: 64 KB).

Two further options. In the course of this work we also added `-clock seconds`, under which the timeline's snapshots are taken every N milliseconds of wall time and every row carries a `Seconds` column, and `-footprint pages`, which counts the 4 KB pages that hold a touched byte. We use both in Section 11.1.

6 Fitting reservations

`sites.py` reads the blocks of one or more profiling runs, keeps the blocks of at least 64 KB (the ones `libmpr` serves), and fits, per site, the bytes a block will need as a function of its requested size n.

Prefix models. `libmpr` commits a block from its start, so a prefix prediction is safe for a block only if it reaches the block's last touched byte, and the fit uses the highest touched offset. Four models are considered:

- `dead`: 0, when no block of the site was touched;

- `const`: a, when the highest offset does not grow with the request;
- `ratio`: $a\,n$;
- `affine`: $a\,n + b$, the lowest line through two blocks that lies above the covered blocks.

A model *covers* a set of blocks if its prediction reaches each block's highest offset. Among the models that cover blocks holding at least a quantile q of the site's requested bytes ($q = 0.9$ by default), the fit keeps the one that commits the least over the training blocks. A site is left out when its best model would commit at least 95% of what was requested.

Shaped reservations. A block that is used sparsely across its whole length defeats a prefix. llama2.c's KV caches are laid out as [layer][position][dimension], and a run of 128 tokens writes the first rows of every layer, so the highest touched offset lies at 93% of the block while 12.5% of it is used. The `runs` model commits a set of ranges of the block, given as fractions of its size: the union of the training blocks' touched page runs, with the closest ranges merged until at most 128 remain. Windows separates reserving address space from committing it, so that a program can commit only the pages it will use in a reserved region [30], and the `runs` model applies the same split, chosen from a profile. The fit keeps `runs` when it commits less than 70% of the best prefix model and saves at least 64 KB.

Run parameters. A KV cache shows the limit of fitting against the request alone: its size is fixed by the context length, and its use grows with the number of tokens the run processes, which no allocation reveals. A `runs` reservation commits what the profiled runs touched, so a longer run uses memory beyond it (Section 10.7). The `periodic` model is a hybrid of the profile and the run's parameters. The profile gives the shape: every block is K equal periods (one tensor per layer), each touched in a prefix of the same length. The run gives the size: the prefix is fitted as $a\,T + b$ bytes, where T is a run parameter recorded with each profile (`params.txt`, here `tokens`). `libmpr` reads the run's value from `MPR_PARAMS=tokens=`T at start-up and commits the first $a\,T + b$ bytes of each period. With three values of T or more, a and b come from a least-squares line shifted up to cover every profile; with fewer, the line passes through the origin with the largest prefix per unit of T. For llama.cpp with SmolLM2-135M, four profiles (36 to 516 tokens, two context lengths) give $K = 60$, $a = 384$ bytes per token (a row of 192 half-precision values) and $b = 2.5$ KB. The fit prefers `periodic` whenever a site's blocks have its shape, also when it commits more on the training blocks, because only it follows T to longer runs. The value of T has to come from whoever starts the job; for llama.cpp it is the prompt's token count (from the model's tokenizer) plus the tokens to generate, and for llama2.c the number of steps, which is an upper bound because llama2.c stops at the end of a story. This answers, for the KV cache, the limitation that a range reservation transfers only between blocks of the same layout and use (Section 12).

A block whose touched runs start at the same places in every profile but grow with a run parameter gets a `ranges` model: the starts, as fractions of the block, and each run's length, from its start to the end of the run that covers it, fitted as $a\,P + b$ in the same way. llama.cpp's compute buffer is such a block: its 12 runs start at fixed pages, and their lengths follow the prompt's length, at about 26 KB per prompt token, not the number of tokens generated. Several run parameters can be recorded with a profile (`tokens`, `prompt`); the fit tries each and keeps, per site, the one that commits least on the training blocks, which puts the KV cache on `tokens` and the compute buffer on `prompt`. A run's parameter must be the tokens it actually processed: a profile of a run that stopped at an end-of-text token before the requested length, recorded at the requested length, pulled the KV cache's fitted slope from 384 to 175 bytes per token.

Calling contexts. When a site's blocks have caller frames, the fit groups them by their first caller frame, then by their first two, and fits each group separately. It keeps the shallowest grouping

Code	Becomes	Purpose
`malloc`, `calloc`, `realloc`, `aligned_alloc`, `memalign`, `valloc`, `posix_memalign`, every `operator new`	`__mpr_alloc`, `__mpr_calloc`, `__mpr_realloc`, `__mpr_posix_memalign` with the original arguments, a site id and a kind	allocation sites
an allocation function used as a value (stored, passed, in a table)	a thunk with the function's type that calls the runtime with its own site id	indirect calls
`memset(p, 0, n)`, `bzero`, `llvm.memset` with value 0, $n \geq 4$ KB or not constant	`__mpr_zero(p, n, site)`	zeroing
result of `vector<T>::_M_check_len` (libstdc++), `__recommend` (libc++ vector and string)	`__mpr_capacity(len, 0, sizeof(T), point)`	growth point (early)
`basic_string::_M_create(size_type&, size_type)`	the capacity argument, through `__mpr_capacity`	growth point (early)
`_Hashtable::_M_rehash(n, ...)`, libc++ `__rehash(n)`	the bucket count, through `__mpr_capacity` (8-byte buckets)	growth point (early)
a growth point inlined into the program	its id becomes the call site's, from the inlined location chain	capacity per call site
a call from the program to a container member that may grow it	bracketed by `__mpr_growth(id)` and `__mpr_growth(0)`	capacity per call site

Table 2: What `MemPrintResize` rewrites. Calls that free memory are not rewritten: `libmpr` recognises its own blocks.

whose total commitment is lower than the merged fit's by at least 5% of the site's requested bytes. The site then gets one line per context, `site@frame1[@frame2] model a b`, and its merged line, if it saves anything, serves contexts that the profile did not see.

Container capacities. For each capacity key with chains whose final buffer reached 64 KB, the fit writes `key cap A B`: A is the q-quantile of the final buffers' used bytes, and B is the largest final size of the key's small chains (0 when there are none). `libmpr` reserves A bytes once a container asks for more than B, so that the many small containers that a call site may grow keep their usual growth.

Builds. For every image that an address key names, the file gets a line `# image name build-id hex`, read from the image's ELF note.

A profile can come from the same input the program will run on (we call this *own profile*, which measures the mechanism) or from other inputs of the same program (*held out*, which measures how a reservation transfers). The quantile leaves some blocks uncovered by design, and Section 10.15 shows that under-predictions are safe.

7 The `MemPrintResize` compiler pass

`MemPrintResize` is a new-pass-manager plugin for LLVM 21 [19], 574 lines of C++, loaded into clang with `-fpass-plugin`. It registers the same module pass at two extension points of the optimisation pipeline: an early instance at `PipelineStartEP`, before inlining, which handles growth points, and a late instance at `OptimizerLastEP`, after inlining, which handles everything else. It has three modes: `profile` (the default) routes allocations through the runtime, `apply` also embeds a reservation file, and `off` does nothing. Table 2 lists what it rewrites.

7.1 Allocation calls and address-taken allocators

For every call or invoke whose callee is an allocation function, the late instance builds a call to the matching runtime entry point with the original arguments, a site identifier and a kind, replaces every use of the old call, takes over its name and erases it. `operator new` under an `invoke` becomes a plain call followed by a branch to the invoke's normal destination, because the runtime reports failure the way `malloc` does, and the landing pad loses that predecessor.

An allocation function can also reach the program as a value: stored in a variable, passed to a function, or placed in a table of function pointers, as libraries that let the caller choose an allocator do. A call through such a pointer is an indirect call, and it bypasses the rewriting. The pass replaces every use of an allocation function that is not the callee of a call with a thunk, an internal function with the same type whose body calls the runtime with a site identifier of its own, derived from the location of the use. The uses in a constant, such as an initialiser, share one thunk per module. Allocations made inside shared libraries that the build does not compile reach `libmpr` through symbol interposition [35]: `libmpr` defines `malloc` and its siblings, so every library's calls resolve to it, with a return-address site and calling contexts.

7.2 Stable site identifiers

A site identifier must name the same allocation in a profiling build and an apply build, distinguish allocations that the optimiser has inlined into different callers, and survive edits elsewhere in the program. `MemPrintResize` computes a 64-bit FNV-1a hash over the callee's name, the name of the function that contains the call after inlining, and the call's debug location chain from the innermost location outward through every `inlinedAt` entry. An allocation inside an inlined `std::allocator::allocate` therefore resolves to the program line that grew the container. Each level of the chain is hashed as the file's base name, the function's name, the line relative to the first line of the function, and the column. Sample-based profiles key their counts the same way, by the offset from the function's first line, so that a profile survives edits outside the function [6, 25]. The base name keeps the identifier independent of the directory the program is built in. Without debug information the hash falls back to the function's name and the call's position in it, so the build needs `-g` (line tables are enough). Ayupov et al. match stale profiles to a new binary by the structure of its code [3]. We use the simpler line-offset scheme and detect what does not match: `scripts/check_reservations.py` lists the reservations whose identifiers a new build no longer has, from the site map that the pass writes (`0x...  kind file:line function < ...`).

7.3 Container capacities

A `std::vector` that grows by `push_back` doubles its capacity, copies its elements, and frees the old buffer, so for a moment it holds both. libstdc++ computes the new capacity in `_M_check_len`, libc++ in `__recommend`, `std::string` in `_M_create` (which takes the capacity by reference), and a hash table chooses its new bucket count before `_M_rehash`. After inlining these calls no longer exist, so the early instance wraps them before inlining:

```
%len = call i64 @_ZNKSt6vectorI...E12_M_check_lenEmPKc(ptr %this, i64 1, ptr @.str)
%cap = call i64 @__mpr_capacity(i64 %len, i64 0, i64 8, i64 <point>) ; replaces %len
```

`libmpr` returns at least `len`, so the capacity only grows: a wrong prediction cannot shrink a container, and an excess is backed lazily. Three refinements make the prediction precise.

Element size. `__mpr_capacity` receives a count of elements, and the profile measures bytes. The pass reads the element size from the debug information: the function that contains the growth

point is a member of the container's class, whose first template type parameter is the element type, and its size follows from the DWARF type entry through typedefs and qualifiers [7]. A hash table's buckets are pointers (8 bytes), and when the debug information has no parameters the pass passes 1.

One capacity per call site. The growth point sits in a template instantiation, such as `_M_realloc_insert<long>`, which every vector of that type in the program shares. A single capacity for all of them would give the final size of the largest vector to every small one. When the optimiser inlines the growth path into the program's code, as clang at `-O2` did for most `std::vector` growth in our programs, the late instance finds `__mpr_capacity` with an inlined location chain that ends at the program's call, and it replaces the instantiation's identifier with the call site's. When the growth path stays a call, for example `basic_string::_M_mutate` or `_Hashtable::_M_insert_unique_node`, the pass brackets every call from the program's own code to a container member that may grow it (`push_back`, `insert`, `append`, `operator[]` and the libraries' internal growth functions, recognised from the demangled name) with `__mpr_growth(id)` before it and `__mpr_growth(0)` after it, on both edges of an `invoke`. A function is the program's own when its debug information names a file outside the system's C++ headers. `__mpr_capacity` then uses the per-thread call site that `__mpr_growth` set. This is a calling context of depth one, maintained by the program as in probabilistic calling context [5]. The libstdc++ profile mode and Perflint report a vector's suggested initial size to the programmer [11, 24], and an IBM patent sets a Java array's initial size from the highest index accessed per allocation context [34]. Our capacity is set automatically, per call site, and only ever raised.

Small containers. A call site can grow many small containers and a few large ones. The `cap` model reserves the final size only once a container asks for more than the largest size that the call site's small containers reached in the profile.

7.4 Zeroing

The late instance replaces `memset(p, 0, n)`, `bzero` and the `llvm.memset` intrinsic with a zero value by `__mpr_zero(p, n, site)` when n is at least 4 KB or not a constant. A call that the compiler would have emitted as a `memset` library call already reaches `libmpr`'s `memset`, but code built with `_FORTIFY_SOURCE` calls `__memset_chk` instead [9], and a static binary has no symbol to interpose. Smaller constant clears stay as they are, because the compiler expands them into stores.

7.5 Embedding reservations

In apply mode the late instance reads the reservation file and adds it to the module as a constant string, `__mpr_embedded_reservations`. Every translation unit carries a copy, so the global has `weak_odr` linkage in a COMDAT group of the same name, and the linker keeps one. (`weak_any` kept 48 copies in darknet's shared library, and `linkonce_odr` was discarded as unreferenced.) `libmpr` declares the symbol weak and reads it at start-up when `MPR_RESERVATIONS` is not set, so an apply build needs no environment.

7.6 Toolchain notes

clang parses `-mllvm` options before it loads a `-fpass-plugin`, so the plugin's options are not yet registered when they are parsed. The pass therefore reads `MPR_PASS_MODE`, `MPR_PASS_RESERVATIONS` and `MPR_PASS_SITEMAP` from the environment, and the same settings remain available as options under `opt`. Our clang installation carries a configuration file that puts a newer GCC's headers first, so builds pass `-gcc-install-dir` for the system GCC 8, whose libstdc++ the programs run with,

and `-no-default-config` where that newer GCC's intrinsics headers break the build. A build script (`scripts/resize_build.sh`) builds the plugin and `libmpr`, sets these variables, links with `-build-id`, and runs the program's own build command. Five `opt`-plus-FileCheck tests check the rewriting of allocation calls, growth points of the three container kinds, zeroing and thunks, and end-to-end tests build programs with the pass, profile them and run them with their reservations.

8 The `libmpr` runtime

`libmpr` is about 1,000 lines of C. It defines `malloc`, `calloc`, `realloc`, `reallocarray`, `free`, the aligned allocators, `malloc_usable_size`, the `operator new` variants, `memset`, `__memset_chk` and `bzero`, so it serves both a preloaded binary and a binary built with `MemPrintResize`. A block smaller than `MPR_MIN` (64 KB) or at a site without a reservation goes to glibc through `__libc_malloc` and its siblings, unchanged.

8.1 Finding a block's reservation

A request names its site by return address (preloaded) or by identifier (pass). When the reservation file has context lines for that site, `libmpr` takes the frames above the call with glibc's `backtrace()`, finds the call's own return address among them, and looks up the deepest context line that matches the next frames, then shallower ones, then the site's own line. `backtrace()` loads `libgcc_s` on its first use, which allocates, so `libmpr` calls it once at start-up and guards against reentry. Only sites with context lines pay for a backtrace, and only for large requests.

8.2 Serving a block

For a large block at a reserved site, `libmpr` creates one anonymous mapping of the full size with `MAP_NORESERVE` and maps the predicted part again with `MAP_FIXED` and without `MAP_NORESERVE`, which makes it accountable: the first p bytes for a prefix model, or each range of a `runs` model. The program receives a pointer to all n bytes. If it touches beyond the prediction, the kernel backs the page as for any anonymous memory, with no signal the program could observe. A design that protected the remainder with `PROT_NONE` and grew the block on a fault would also break system calls, because a `read()` into a protected page returns `EFAULT` instead of faulting. A `dead` site's block is entirely `MAP_NORESERVE`. A large `calloc` needs no `memset`, because fresh anonymous pages are zero. Blocks live in an open-addressing table keyed by address and in a sorted array of start addresses, which finds the block that holds an arbitrary address. `free` recognises a runtime block by its page alignment and a table lookup, records its resident bytes with `mincore` for telemetry, and unmaps it. The tables are themselves mapped `MAP_NORESERVE`, so `libmpr` commits nothing of its own.

8.3 Growing and shrinking

A runtime block grows without copying where possible. `libmpr` first tries to map the new tail directly after the block with `MAP_FIXED_NOREPLACE`. If the address is taken, it reserves a new range and moves the block's two mappings into it with `mremap(MREMAP_FIXED)`, which moves page tables instead of data. Growth is geometric, at least 1.5×, so a table grown one element at a time does not remap at every step, and a shrink is kept in place unless it halves the block. We found two kernel behaviours on the way. The kernel keeps a tail that was added in place and written separately as its own mapping, and `mremap` fails with `EFAULT` on a range that spans several mappings. A

failed `mremap(MREMAP_FIXED)` has also already unmapped its destination. Open MPI's start-up (in `libpciaccess`) grows a table one element at a time past the size threshold and crashed on the first version. `libmpr` now maps the destination again and copies such a piece. A block committed in ranges consists of many mappings, so it grows by a new block and a copy.

8.4 Container capacities

`__mpr_capacity(len, size, elem, point)` looks up the reservation of the call site that `__mpr_growth` set on the thread (`cap:0x<site>`), else of the growth point (`id:0x<point>`, which for an inlined growth point is already the call site's). For a `cap A B` model it returns $\max(\texttt{len}, \lceil A/\texttt{elem} \rceil)$ once $\texttt{len} \cdot \texttt{elem} > B$, and `len` otherwise.

8.5 Skipping the zeroing of untouched pages

Programs that clear a buffer they have just allocated make every page of it resident. When `memset`, `__memset_chk`, `bzero` or `__mpr_zero` clears at least 16 KB inside a runtime block, wherever in the block it starts, `libmpr` reads the range's entries from `/proc/self/pagemap` [21] and writes zeros only to pages that need them:

- a page that is neither present (bit 63) nor swapped (bit 62) has never been touched since the block was mapped, and it reads as zero, so it is skipped;
- a page that is present but not mapped exclusively (bit 56 clear) is the kernel's shared zero page, mapped by an earlier read, or a page shared with a forked child, so `libmpr` reads it and skips it if it is all zeros, which is correct in both cases;
- every other page, and the partial pages at both ends, are written.

Other `memset` calls go to glibc's implementation, found with `dlsym(RTLD_NEXT)` at start-up (a byte loop that GCC must not turn back into a `memset` serves the calls made before that).

The first version checked only the present bit. llama.cpp clears its KV cache a second time after a warm-up decode, and attention had read the whole cache by then, so every page mapped the zero page and looked present. After the zero-page check, 123 MB of the cache were still resident. The cause was transparent huge pages, which this machine enables for all anonymous memory: the warm-up writes a few rows into each layer's K and V tensors (about 3 MB each), and each first write faults in a 2 MB page [23]. `libmpr` therefore marks the pages whose zeroing it skipped `MADV_NOHUGEPAGE`. Those are the pages it chose to leave untouched, and a huge page would undo the choice. Pages the program has touched keep huge pages. `MPR_THP=0` and `MPR_THP=1` disable huge pages for all runtime blocks or leave them as the system sets them, and `MPR_ZERO=0` turns the zero skipping off.

8.6 Builds

When the reservation file records an image's build identifier, `libmpr` reads the build identifier of the image it has loaded from its `PT_NOTE` segment, the first time it makes a key in that image. If the two differ, or the loaded image has none, it ignores that image's address keys and says so once on standard error. The pass's identifiers do not depend on the build and are not checked.

8.7 Threads' stacks

glibc maps a thread's stack, 8 MB by default, as one committed region when the thread is created, although a thread touches only as deep as it calls. `libmpr` wraps `pthread_create`: a thread created without a stack of its own gets one mapped `MAP_NORESERVE`, below a guard page, with the caller's other attributes (detach state, scheduling, affinity) copied, and `pthread_join` unmaps it. A stack is a reservation like a block's, of zero committed bytes, and its pages are backed as the thread touches them, so a deep recursion still works. A detached thread's stack stays mapped until exit, because no join says when it is free.

8.8 Profiling at native speed

The reservations need a profile of each large block's touched pages, which `libmpr` can measure itself. With `MPR_PROFILE=`*dir*, it serves every large block, keeps transparent huge pages off them, and when a block is freed (or at exit) writes the block's site, its two caller frames, its size, touched pages, high water and runs of touched pages, read with `mincore`, in the Pin tool's `-sites` format, and the loaded images with their paths for the build identifiers. A block's pages are fresh when `libmpr` serves it and `libmpr` skips the zeroing of untouched pages, so its resident pages are the pages the program touched; the profile is exact to the page, which is what a reservation is made of, and it costs one run at native speed. `sites.py` reads it like a Pin profile. It differs from Pin's in two ways: a block grown by `realloc` is one row at its final size, where the Pin tool records every buffer of the chain, and small blocks and container growth chains are not profiled.

8.9 Telemetry and settings

With `MPR_REPORT`, `libmpr` writes per site at exit the number of blocks, the bytes requested, predicted (committed), resident at free, the blocks that used more than predicted, and the zeroing it skipped. `__mpr_committed()` returns the predicted committed total for a scheduler to read. `MPR_SCALE` multiplies every prediction for robustness tests, and `MPR_ALL` serves every large block, reserved or not, to measure the mechanism alone. Two lock-related fixes were needed: `dladdr`, used to name a return-address site, is called outside `libmpr`'s lock, and the exit handler releases the lock before it writes the report, because `fclose` frees its buffer through `free`.

9 Per-site profiles from source

MemPrint has a second route to a footprint, which does not run the program: an abstract interpreter over the clang AST that executes the program's source with concrete sizes and vectorised loops, charges every reference to the object it touches, and reports the bytes touched [31]. Its result was a footprint for the whole program, not per-site data, so the reservations above came from Pin. We extended the interpreter to produce the same per-site profile as the Pin tool, in the same files, so that `sites.py`, `MemPrintResize` and `libmpr` take it unchanged.

Sites and contexts. The interpreter records, for each heap object it creates (a `malloc`-family call or a `new` expression), the source location of the allocating expression and the locations of the calls that entered the functions above it, innermost first. Each interpreter frame keeps the call expression that entered it; constructor calls, including delegating constructors, enter frames without going through the interpreter's call path, and had to push their construct expressions explicitly, or a `pvector`'s buffer would have been attributed to the caller of the function that built the `pvector`.

Vectors. The interpreter models `std::vector` instead of running libstdc++, and places every vector's elements in one arena object. Each buffer the model creates (construction, `push_back`,

`resize`, growth) is recorded with its arena range, the call that made it and the buffer it replaced, which links the buffers of one vector into a growth chain, as the Pin tool does at run time.

Measures. From the references the interpreter charged to each byte, a block's touched bytes, high water, touched pages and page runs follow as the Pin tool measures them. The module writes `Static_<name>_blocks.csv` and `_capacity.csv`.

Keys. A static site is a chain of source locations, and a reservation needs the key the running program will present. With the site map of the build that will run, which lists each identifier's allocation call and the locations it was inlined at (Section 7.2), each block gets the identifier whose location chain is the longest prefix of the block's chain, among identifiers of the same allocation function (a `realloc` and a `malloc` on the same line are different sites). Locations inside the C++ library's headers are skipped, since the interpreter models those functions rather than running them, and a vector's buffer is matched to an identifier whose chain passes through the library function that its operation calls (`push_back` to `_M_realloc_insert`, `resize` to `_M_default_append`). Container chains are keyed by the growth context (`cap:`) or the inlined growth point (`id:`), as the Pin tool keys them. A static calling context beyond what the identifier inlined is dropped, because `libmpr` matches further frames as return addresses, which a static profile does not know.

10 Evaluation

10.1 Setup

All runs used a two-socket Intel Xeon E5-2620 v4 (16 cores, 32 threads) with 62 GB of RAM, Linux 4.18 (RHEL 8) with heuristic overcommit and transparent huge pages set to `always`, GCC 8.5 and clang 21. Each configuration ran three times, and we report the mean of the peak RSS and the peak commit, sampled every 20 ms for the program and its children. Outputs were compared by hash: all printed lines except those that report a time or a rate, the modularity line for miniVite, the predictions for darknet, and the generated text for the LLM engines.

The workloads are:

- darknet AlexNet inference on 4 images [39], built with GCC (preloaded) and with `MemPrintResize`;
- miniVite [13] with 32768 vertices and 4 OpenMP threads, built with GCC (preloaded) and with `MemPrintResize`;
- llama.cpp b11500 [12], CPU only, 4 threads, greedy decoding of "Once upon a time" with SmolLM2-135M-Instruct (32 and 128 tokens) and SmolLM2-360M-Instruct (64 tokens), both quantised to Q8_0 [1], at the model's default 8192-token context, built with GCC (preloaded) and with clang and `MemPrintResize`;
- llama2.c [16], greedy, 128 steps, with the stories15M, 42M and 110M models;
- the 23 programs of the scheduling corpus [32] at their largest inputs: PolyBench kernels, GAP, HPCCG, LULESH, miniFE, Kripke, Quicksilver, NPB, darknet, miniVite, the Lua, Perl and Python interpreters, SQLite and GCC's cc1.

Each input has an *own* profile and a *held-out* profile from the program's other inputs (for darknet, the reference and Extraction networks; for miniVite, 8192 and 16384 vertices). The profiles use the Pin tool with `-sites`, which includes calling contexts and `-skip_zeroing`. We call the system as first built (prefix models only, one site per call, the first zero-skipping runtime) the *first version*, and

	glibc		libmpr, own profile		libmpr, held out		Output
Workload	RSS	Commit	RSS	Commit	RSS	Commit	identical
darknet AlexNet	287.3	534.7	287.7	284.6	287.8	519.1	yes
miniVite 32768 (pass)	30.3	46.2	27.0	26.5	28.6	32.2	yes
llama.cpp 135M, 32 tok	380.0	404.5	175.1	55.6	175.2	56.6	yes
llama.cpp 135M, 128 tok	380.1	404.5	177.4	56.4	177.5	56.2	yes
llama.cpp 360M, 64 tok	768.2	560.5	419.0	56.3	424.4	57.4	yes
llama2.c 15M	67.0	6.2	67.4	4.3	67.4	3.3	yes
llama2.c 42M	201.0	36.2	175.9	6.8	175.8	21.6	yes
llama2.c 110M	514.6	78.2	452.5	12.1	469.6	45.8	yes

Table 3: Peak RSS and peak commit in MB (mean of 3 runs) with the refined system. miniVite's glibc column is the GCC build; its libmpr columns are the `MemPrintResize` build. All other libmpr columns are `libmpr` preloaded into the GCC build.

	Peak commit (MB)			Peak RSS (MB)		
Workload	glibc	first	refined	glibc	first	refined
llama.cpp 135M, 32 tok	404.5	265.1	55.6	380.0	193.2	175.1
llama.cpp 360M, 64 tok	560.5	419.9	56.3	768.2	435.4	419.0
llama2.c 42M	36.2	32.8	6.8	201.0	197.6	175.9
llama2.c 110M	78.2	72.8	12.1	514.6	486.8	452.5
miniVite 32768 (pass)	34.0	26.1	26.5	30.3	27.0	27.0

Table 4: The numbers of Figure 3 (MB, mean of 3 runs).

the system with calling contexts, shaped reservations, zeroing left out of the profile, and per-call-site container capacities the *refined* version.

10.2 Headline results

Table 3 and Figure 2 summarise the results. With its own profile, `libmpr` reduces peak commit by 86.3% on llama.cpp with the 135M model (404.5 to 55.6 MB) and by 90.0% with the 360M model (560.5 to 56.3 MB), and it reduces peak RSS by 53.9% and 45.5%. It reduces commit by 84.5% on llama2.c 110M (78.2 to 12.1 MB) and by 46.8% on darknet. Held-out profiles do as well on llama.cpp, where the sites are the same code paths at every model size. Every run of every workload produced glibc's output, and run time stayed between 0.93× and 1.02× of glibc.

10.3 Effect of the refinements

Figure 3 and Table 4 compare the first and the refined version. The refinements reduce llama.cpp's commit by a further 79% (265.1 to 55.6 MB) and llama2.c 110M's by 83% (72.8 to 12.1 MB). Three refinements combine on llama.cpp:

- **Zeroing left out of the profile.** The first profile counted the start-up `memset` as touching the whole 189 MB KV cache, so the cache was committed in full. With `-skip_zeroing`, the profile sees the rows the model writes.
- **Calling contexts.** The KV cache, the compute buffer and several small buffers are allocated through one site in `ggml_aligned_malloc`. The fit splits that site by two caller frames, and each buffer gets its own model.

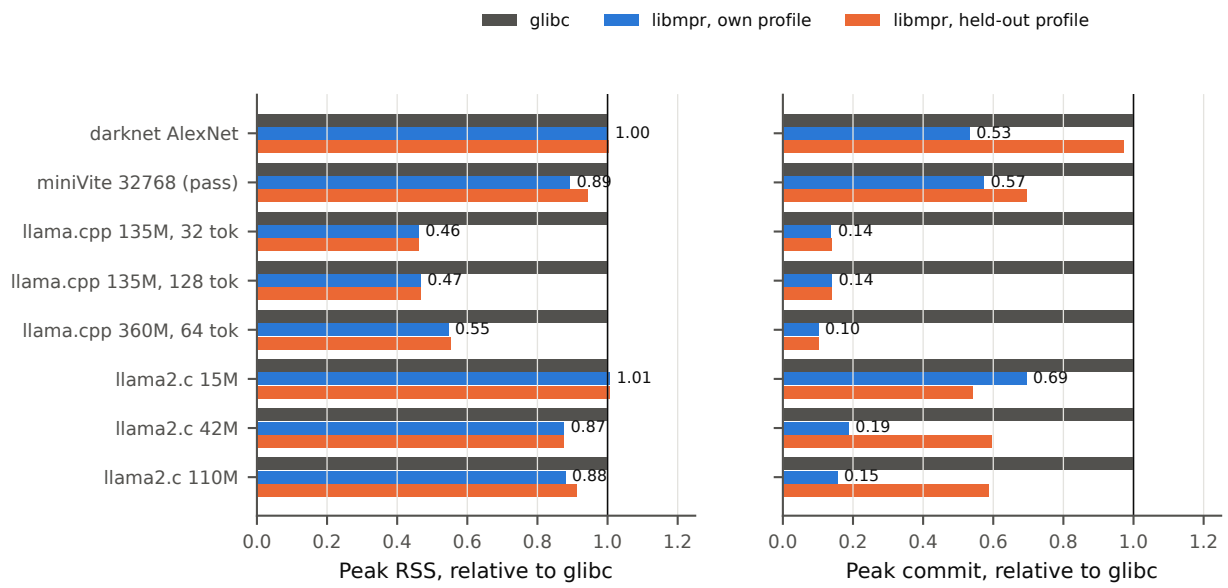


Figure 2: Peak RSS and peak commit under `libmpr`, relative to glibc. Every bar is a run whose output was identical to glibc's.

- **Shaped reservations.** The KV cache is 60 tensors (30 layers, K and V), each written in its first rows. Its `runs` reservation commits 1.0 MB of the cache, against 189 MB for any prefix, and the compute buffer's commits 0.4 MB of 115 MB.

On llama2.c, the shaped reservation alone does the work: each KV cache gets one committed range per layer. RSS falls further too (llama2.c 110M: 486.8 to 452.5 MB), because ranges committed as separate mappings no longer form transparent huge pages across the untouched rows between them. On miniVite, the refined version matches the first version's hand-made capacity, with capacities fitted automatically per call site (Section 10.5).

10.4 darknet: dead training buffers

Figure 4 shows what the profile found. darknet's largest site allocates the `weight_updates` buffers of its connected layers (`connected_layer.c:35`) and convolutional layers (`convolutional_layer.c:196`), which only training uses, so inference never touches them. With its own profile, `libmpr` maps them `MAP_NORESERVE`, and peak commit falls from 534.7 MB to 284.6 MB, close to the 270 MB the program touches. RSS does not change, because the untouched buffers were never resident. The `MemPrintResize` apply build, with the reservations embedded and no environment, gives the same 284.6 MB. A held-out profile from the reference and Extraction networks saves only 15.6 MB, because those networks have no fully connected layers: the profile has no data about the connected layers' site, which holds 234 MB of AlexNet's untouched buffers, and only the convolutional layers' 15 MB carry over. A profile helps the code paths it covers, and no refinement changes that.

10.5 miniVite: container capacities and where its memory goes

miniVite touches 15.4 MB at its peak but holds 28.7 MB of RSS at its peak RSS and commits 44.0 MB at its peak commit (Figure 5, left). The breakdown, from `/proc/<pid>/smaps` at each

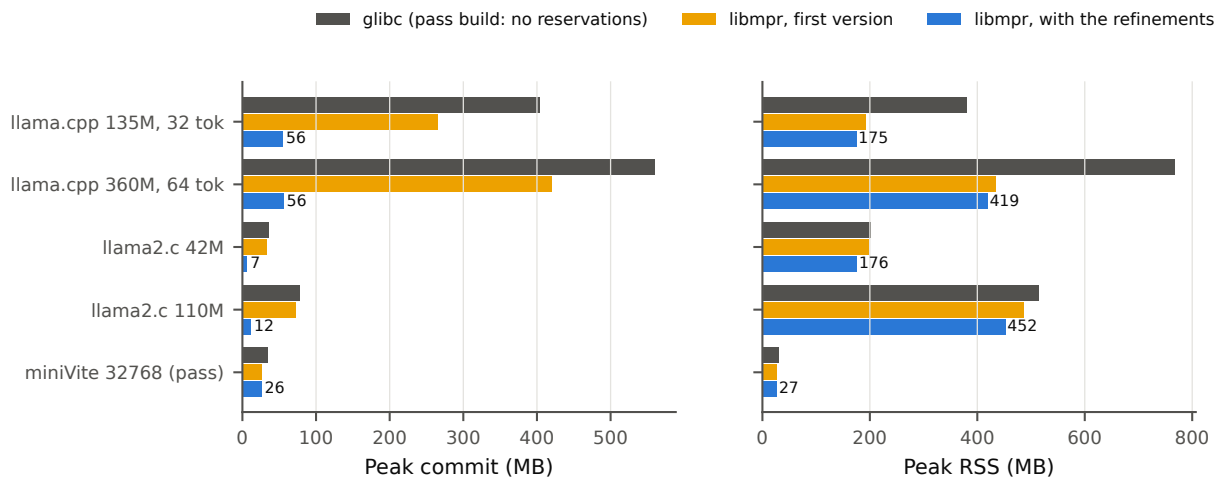


Figure 3: Peak commit and peak RSS under glibc, the first version of the system and the refined version, with own profiles. miniVite is the `MemPrintResize` build; its glibc bar is the build without reservations.

peak, explains the gap:

- **Code, not data, is a third of the RSS.** 9.6 MB is library and program code (Open MPI, libstdc++, libgomp), which MemPrint does not count because it measures data references, not instruction fetches.
- **Vector growth.** `edgeList` (`graph.hpp:777`) grows by `emplace_back` without `reserve`, and it is the only large heap site live at the peak. When it doubles, the old 6.3 MB buffer and the new 12.6 MB buffer are both live (18.9 MB requested, 12.8 MB touched), and huge pages back the new buffer's 6.5 touched MB with about 8 MB.
- **Small blocks pay glibc's per-chunk overhead.** 1.8 million blocks smaller than 64 KB (hash table nodes of 24 to 47 bytes and small vectors) request 57 MB over the run but occupy 81 MB of glibc chunks (1.42×), and the heap holds them in 10.4 MB at the peak.
- **Untouched library memory.** Open MPI's start-up allocates 1.6 MB that it never touches.
- **Commit is mostly thread stacks.** 32 MB of the peak commit is four 8 MB OpenMP thread stacks, of which 2 MB is resident. A smaller `OMP_STACKSIZE` would remove that commit, which no allocator change can.

Built with `MemPrintResize`, miniVite has 20 growth points, 23 of them inlined into call sites in its own code, and 44 calls to container members bracketed as growth contexts. The profile of the 32768-vertex input yields 12 capacities: `edgeList`'s final size (6.5 MB, one allocation instead of eight totalling 25 MB) and those of 11 vectors whose growth stays out of line, reached through growth contexts. Two call sites, a vector and a hash table, each grow about half a million small containers and get no capacity, because their chains never reach 64 KB. With its own profile, peak RSS falls from 30.3 MB to 27.0 MB (11%) and peak commit from 34.0 MB to 26.5 MB (22%), with the same modularity and run time (1.79 s against 1.80 s). The held-out profile from 8192 and 16384 vertices reserves the 16384-vertex final size, half of what the 32768-vertex edge list needs, and saves less (28.6 MB RSS, 32.2 MB commit).

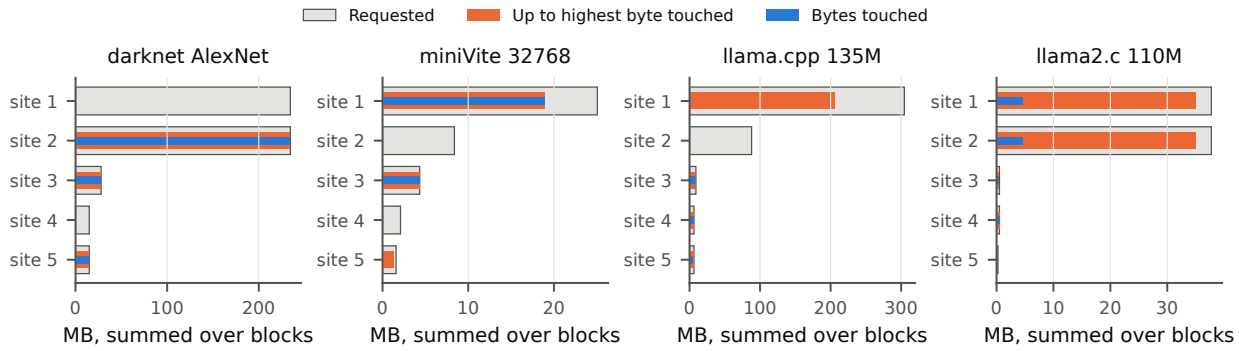


Figure 4: The five largest allocation sites of four workloads, summed over their blocks: bytes requested, bytes up to the highest touched offset (what a prefix reservation must commit), and bytes touched, from the refined profiles. darknet's first site is dead. llama.cpp's first site is `ggml_aligned_malloc`, which serves the KV cache and the compute buffer; with zeroing left out, few of its bytes count as touched. llama2.c's two KV caches are touched sparsely across the whole block.

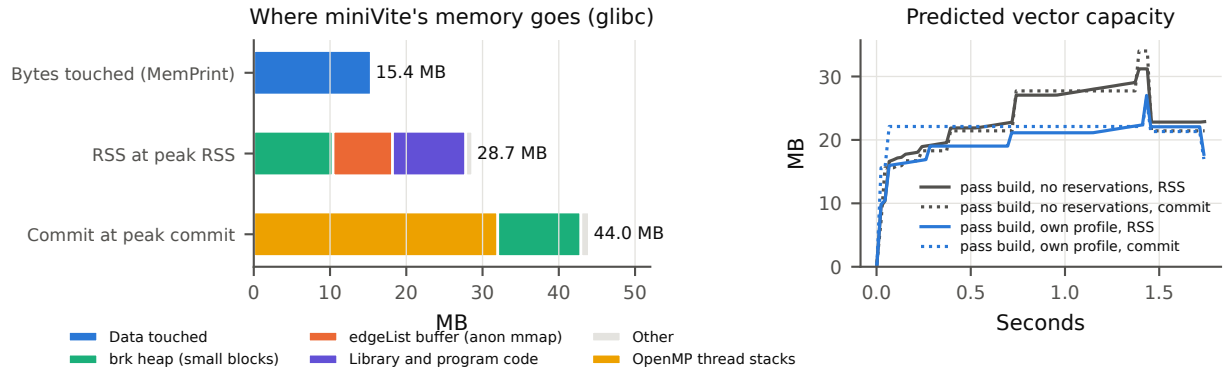


Figure 5: Left: miniVite's bytes touched, its RSS at peak RSS and its commit at peak commit under glibc, by mapping (smaps, 32768 vertices, 4 threads). Right: RSS and commit over time of the `MemPrintResize` build without reservations and with its own profile.

The first measurement of the refined pass ran 15% slower with reservations. The cause was `libmpr`'s capacity lookup, which formatted a key, took a lock and searched the site list at every growth of every container, about a million times in this run. A table by numeric identifier, filled when the reservations load and read without the lock, removed the cost, and the build without reservations became 17% faster as well (2.17 to 1.80 s).

10.6 LLM inference

llama.cpp. With the 135M model, llama.cpp allocates three large buffers through `ggml_aligned_malloc`: a 188.7 MB KV cache for 8192 tokens, a 115 MB compute buffer of which 0.3 MB is touched, and two 44 MB scheduler arrays (`ggml_backend_sched_new`) that are never touched. It clears the whole KV cache with `memset` at start-up and again after its warm-up decode. Under glibc, the whole cache is therefore resident although a 32-token completion uses 0.4% of it. The model's weights are mapped from the GGUF file, so their 143 MB count in RSS but not in commit.

RSS and commit fall for different reasons (Figure 6). RSS falls from 380 MB to 175 MB be-

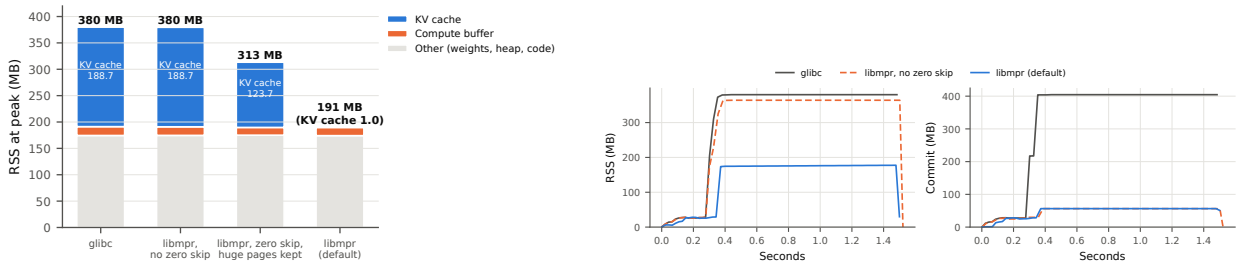


Figure 6: llama.cpp with SmolLM2-135M. Left: RSS at the peak by buffer, measured from `/proc/<pid>/pagemap` over each buffer's address range (32 tokens, first version of the runtime; "other" is the mapped weights, the heap and code). Right: RSS and commit over a 128-token run with the refined system.

cause of the zero skipping. To attribute it, we recorded each buffer's address and measured its resident pages at the peak from `/proc/<pid>/pagemap` under four configurations of the first runtime. Without zero skipping, the KV cache holds 188.7 MB, as under glibc. Skipping the zeroing alone leaves 123.7 MB, all of it huge pages faulted in by the warm-up's writes of one row per layer. Skipping the zeroing and keeping huge pages off the skipped pages leaves 1.0 MB. The compute buffer holds 15 to 17 MB under every configuration, which is the part llama.cpp uses at the peak, and the remaining 174 MB (weights, heap and code) does not change. Commit falls from 404.5 MB to 55.6 MB because of the reservations (Section 10.3): the scheduler arrays are `dead`, and the KV cache and the compute buffer are committed in ranges. Run time does not change (0.71 s against 0.73 s for 32 tokens, 1.81 s against 1.95 s for the 360M model).

llama2.c. llama2.c's KV caches are also sized for the full sequence (37.7 MB each for the 110M model, of which 4.7 MB is touched for 128 of 1024 positions). It allocates them with `calloc` and does not clear them, so their untouched part was never resident under glibc either. They are laid out as [layer][position][dimension], so every layer's first rows are touched and the highest touched offset lies at 93% of the block (Figure 4). A prefix reservation must cover it: the first version committed 72.8 of 78.2 MB. The shaped reservation commits one range per layer, 12.1 MB in all. The held-out profile, from models with 6 and 8 layers, places its ranges where a 12-layer model's are only in part, and commits 45.8 MB.

10.7 Unseen inputs and run parameters

A profile describes the input it was taken on. We ran both engines on 21 inputs that no profile saw, with the reservations from each model's default-input profile (Table 5). All 189 runs produced glibc's output. llama.cpp's commit stays at about 56 MB on every input, 78 to 90% below glibc's, and its RSS falls by 24 to 53%. llama2.c's falls by about 80% for the 42M and 110M models. Profiling the new input itself gains nothing: for llama.cpp at 512 tokens the input's own profile commits 54.9 MB (56.0 per-model), at context 2048 56.2 MB (55.7), and for llama2.c 42M at 512 steps 10.0 MB (7.7), because its ranges cover the rows the longer run writes. Pooling the profiles of all models of an engine costs little for llama.cpp (56.3 to 57.8 MB) and much for llama2.c (49 MB instead of 13 MB for 110M), whose models have 6, 8 and 12 layers, so the ranges of one fall elsewhere in another's blocks.

The commit, however, stops describing the run when the run is longer than the profile. The per-model reservation of llama.cpp's KV cache covers the rows of a 36-token run, so a 1,024-token run writes 23.4 MB beyond it, and llama2.c 110M writes 27.2 MB beyond it at 1,024 steps (Figure 7).

		glibc		per-model (`runs`)			periodic, T from the run		
Input	T	RSS	Commit	RSS	Commit	Beyond	RSS	Commit	Beyond
llama.cpp 135M, 16 tokens	20	380	404.6	176	56.0	1.4	176	56.7	1.4
llama.cpp 135M, prompt 2	37	380	404.6	176	56.0	1.9	176	57.2	1.4
llama.cpp 135M, prompt 3	40	380	404.6	176	56.0	1.9	176	57.2	1.4
llama.cpp 135M, 128 sampled	132	380	404.6	179	56.0	4.1	179	59.1	1.4
llama.cpp 135M, 136-token prompt	168	383	404.5	185	56.0	10.6	184	60.1	6.3
llama.cpp 135M, 256 tokens	260	380	404.6	181	56.0	7.0	181	62.1	1.4
llama.cpp 135M, 512 tokens	516	379	404.6	187	56.0	12.9	187	68.0	1.4
llama.cpp 135M, 1024 tokens	1032	380	404.6	199	56.0	24.7	199	80.0	1.4
llama.cpp 135M, context 2048	68	233	257.5	179	55.7	4.9	177	57.6	1.4
llama.cpp 135M, context 4096	68	282	307.1	177	55.8	2.8	177	57.6	1.4
llama.cpp 360M, 256 tokens	261	768	560.4	428	56.7	11.1	428	66.9	1.4
llama.cpp 360M, context 4096, long prompt	264	605	388.3	441	56.3	25.0	440	66.9	13.5
llama2.c 15M, 32 steps	32	66	6.3	66	5.2	0.0	66	3.6	0.0
llama2.c 15M, 128 sampled	128	67	6.2	67	4.8	0.0	67	5.0	0.0
llama2.c 15M, 256 steps	256	68	6.2	69	5.2	1.3	69	6.9	0.0
llama2.c 42M, 64 steps	64	201	36.4	174	7.5	0.0	174	5.5	0.0
llama2.c 42M, 512 steps	512	201	36.2	178	7.3	2.4	178	20.2	0.0
llama2.c 42M, 1024 steps	1024	201	36.2	179	7.7	2.7	204	36.9	0.0
llama2.c 110M, 256 steps	256	507	78.2	458	12.7	4.9	458	22.5	0.0
llama2.c 110M, 512 sampled	512	507	78.2	467	12.8	14.3	459	41.4	0.0
llama2.c 110M, 1024 steps	1024	515	78.2	480	12.5	27.2	517	78.9	0.0

Table 5: The LLM engines on 21 inputs that were never profiled: other prompts, lengths, context sizes and sampling (temperature 0.8). Peak RSS and peak commit in MB, mean of 3 runs; "Beyond" is what `libmpr`'s report found resident beyond the commitment in the reserved blocks at their release, in MB. Per-model: the `runs` reservations from the model's default-input profile. Periodic: the `periodic` reservations, with T given to the run in `MPR_PARAMS`; for llama.cpp T is the prompt's tokens plus the tokens to generate, for llama2.c the steps (at most the model's sequence length). The three configurations ran interleaved. Every run produced glibc's output; time relative to glibc was 0.96 to 1.08 (per-model) and 0.88 to 1.07 (periodic).

This is safe, since the kernel backs those pages on touch, but a scheduler reading the commit would under-count the job. The `periodic` reservation (Section 6) removes it. With T from the run, no KV cache was used beyond its commitment on any input, except 0.25 MB on the 136-token prompt, and the commit grows with the run: 56.7 MB at 20 tokens, 62.1 at 260, 68.0 at 516 and 80.0 at 1,032 for llama.cpp 135M, against 405 MB under glibc. RSS does not change, since the same pages are touched either way. The four profiles are not needed: fitted from the 32-token profile alone (through the origin, 455 instead of 384 bytes per token), the reservation commits 56.2 to 83.8 MB on the same inputs, 3.7 MB more than the four-profile fit at 1,032 tokens, and again no KV cache was used beyond its commitment, with identical outputs. For llama2.c the parameter is the number of steps, which bounds the tokens from above, because a story usually ends after about 200 tokens. A 1,024-step run therefore commits the whole cache (78.9 MB for 110M, glibc's 78.2 MB plus the page rounding of 12 periods), where the per-model reservation commits 12.5 MB and the run uses 27.2 MB beyond it; at 256 steps the periodic reservation commits 22.5 MB and covers the run exactly.

What remained beyond the commitment was outside the KV cache: 1.35 MB at a small site whose `const` reservation came from one profile, and llama.cpp's compute buffer, whose used ranges grow with the number of prompt tokens processed in one batch, not with the run's length (4.3 MB beyond its ranges with the 136-token prompt, 11.7 MB for the 360M model). The profiles had prompts of 4 to 8 tokens, so the fit had nothing to relate it to.

With ten profiles taken under `libmpr` (Section 8.8), at prompts of 4 to 279 tokens and 8 to 512 generated tokens, each in under 1.4 s, and the prompt's length as a second run parameter,

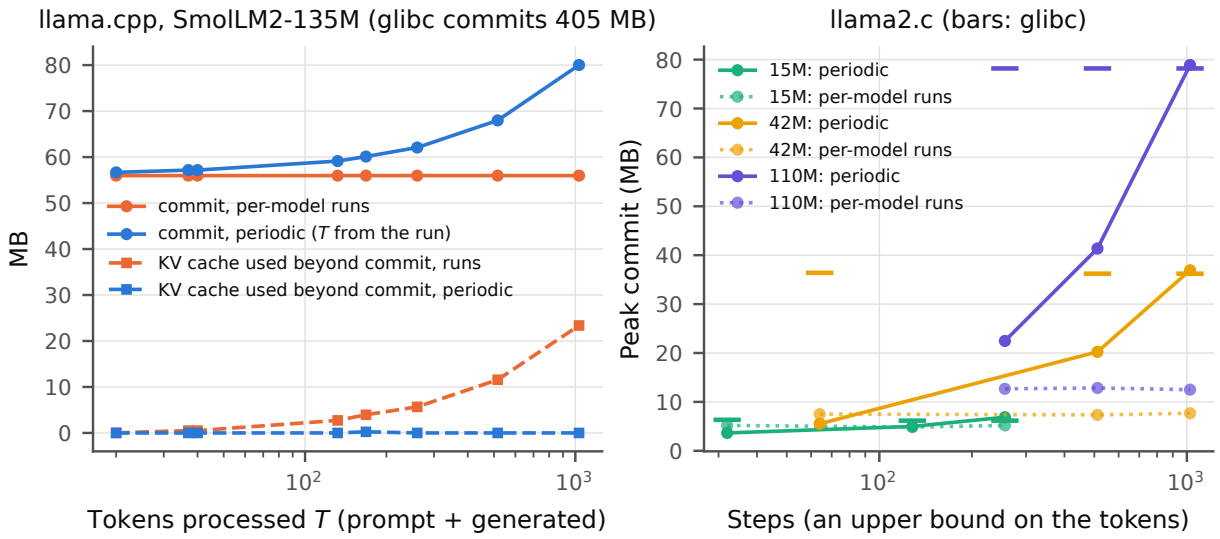


Figure 7: Commit against the tokens a run processes. Left: llama.cpp 135M at its full 8192-token context; the per-model `runs` reservation commits the same at every length and the KV cache outgrows it, the `periodic` one follows T. Right: llama2.c, whose steps bound the tokens from above (a story ends earlier), so at the full sequence length the periodic reservation commits what glibc does.

nothing was used beyond the commitment on nine of the ten unseen inputs of the 135M model (Table 6). The 136-token prompt commits 63.9 MB instead of 60.2 and uses nothing beyond it (6.28 MB before), and RSS falls by 1 to 2 MB. The exception is the 2048-token context, 2.0 MB beyond: its compute buffer is laid out differently, and a range given as a fraction of the block does not carry over between layouts.

10.8 An LLM server

The LLM runs above are single short completions, which use a small part of a KV cache sized for the whole context. A server runs many sequences at once and fills more of its cache as it goes. We ran llama.cpp's parallel example, which serves 8 requests per client from 4, 8 or 16 concurrent clients with continuous batching, for both models at 8192- and 16384-token contexts (Table 7). The reservations come from `libmpr`'s own profile (Section 8.8) of a smaller load, 2 clients and 8 requests, taken in 9 to 17 s, so they are held out from the load they serve. Every run served every request with the same token counts as under glibc. `libmpr` cuts peak commit by 29 to 42% at the 8192-token context and by 18 to 34% at 16384 tokens, where more of the larger cache is written, and its run time is within 0.99 to 1.07 times glibc's. The resident memory falls by less, and less as the server gets busier: by 8 to 14% with 4 clients, 4 to 8% with 8 and 3 to 4% with 16, because the clients' sequences touch the cache that a single completion leaves untouched, and that memory is in use. With 16 clients in an 8192-token context llama.cpp itself ran out of KV-cache slots, under glibc and `libmpr` alike, and served only part of the load.

10.9 Built with the pass, or preloaded

Table 8 compares the two ways of running llama.cpp. They save the same: with its own profile the clang build with `MemPrintResize` commits 56.0 MB and holds 177.5 MB (135M, 32 tokens), against 55.6 MB and 175.1 MB for the preloaded GCC build, and held-out profiles give the same in both.

	Commit (MB)		Beyond the commitment (MB)		
Input	tokens	+ prompt	tokens	+ prompt	RSS (MB)
16 tokens	56.8	56.8	1.35	0.00	175
prompt 2	57.3	57.3	1.35	0.00	175
prompt 3	57.3	57.4	1.35	0.00	175
128 sampled	59.2	59.5	1.35	0.00	177
136-token prompt	60.2	63.9	6.28	0.00	182
256 tokens	62.2	62.4	1.35	0.00	180
512 tokens	68.1	68.3	1.35	0.00	186
1024 tokens	80.1	80.5	1.35	0.00	198
context 2048	57.7	58.0	1.35	2.04	178
context 4096	57.7	58.0	1.35	0.00	176

Table 6: llama.cpp 135M on its unseen inputs: the periodic reservations on `tokens` from four Pin profiles, against reservations from ten `libmpr` profiles (Section 8.8) with prompts of 4 to 279 tokens and 8 to 512 tokens generated, fitted on `tokens` and `prompt` (the KV cache periodic on tokens, the compute buffer `ranges` on the prompt). Mean of 3 runs, interleaved with glibc; every output identical.

In llama.cpp the large buffers come from a few allocation functions, which return addresses name as well as the pass's identifiers do, and none of its growing containers reaches 64 KB of capacity in a way that a reservation changes. The pass is needed for container capacities, which come from inside inlined library code, and it gives identifiers that do not depend on the build; a preloaded run needs no rebuild. Table 9 shows what the pass costs: no measurable compile time on darknet and 2.8% on llama.cpp (one build, while other jobs ran). With the previous identifiers, inserting lines at the top of five darknet files changed every identifier in those files (80.9% of all sites unchanged); with line offsets and base file names, only the allocations below a line inserted inside a function change (96.6% unchanged). Rebuilding the program changes its build identifier, and `libmpr` then refuses the profile's address keys, as it did for llama.cpp's calling contexts after a rebuild: the KV cache's context lines no longer applied, and its commit rose to 261 MB until the program was profiled again.

10.10 The corpus: no regressions

Table 10 and Figure 8 show the 23-program corpus, profiled again with the refined Pin tool. Under glibc, the median program commits 1.01× its bytes touched and holds 1.05× in RSS, so there is little for a resizer to take. The exceptions are darknet (1.98× commit), miniVite (3.00×), Quicksilver (1.25×), cc1 and Python (1.13×) and Lua (1.12×). With own profiles, total peak commit falls by 10.7%, from 2,504 MB to 2,236 MB, most of it darknet's, and Perl's commit falls below its bytes touched (0.92×). Held-out profiles save 0.9%. Total RSS does not change (2,290 to 2,292 MB). Every program produced glibc's output in every run, and run time stayed between 0.95× and 1.07× of glibc with own profiles. One regression appeared while we built the refined runtime and was fixed: `libmpr`'s site table, larger once each site can hold 128 ranges, was backed by a 2 MB huge page at its first entry, which added 2.2 MB of RSS to every program until the table was marked `MADV_NOHUGEPAGE`.

10.11 Profiles from source

We profiled GAP's pr, bfs, cc and sssp at scales 16, 17 and 18 with the interpreter and, for comparison, under Pin (Table 11). Each kernel was built with `MemPrintResize` in profile mode,

Model	Context	Clients	Peak RSS (MB)		Peak commit (MB)		Time	Outputs
			glibc	`libmpr`	glibc	`libmpr`		
SmolLM2-135M	8192	4	400	366	421	246	0.99	same
SmolLM2-135M	8192	8	410	389	430	253	1.04	same
SmolLM2-135M	8192	16	llama.cpp ran out of KV-cache slots under glibc and `libmpr` alike					
SmolLM2-135M	16384	4	583	500	600	417	1.01	same
SmolLM2-135M	16384	8	625	575	644	426	1.04	same
SmolLM2-135M	16384	16	605	584	627	432	1.00	same
SmolLM2-360M	8192	4	811	749	590	417	0.99	same
SmolLM2-360M	8192	8	824	791	601	426	1.02	same
SmolLM2-360M	8192	16	llama.cpp ran out of KV-cache slots under glibc and `libmpr` alike					
SmolLM2-360M	16384	4	1,126	1,001	907	732	1.07	same
SmolLM2-360M	16384	8	1,205	1,129	983	808	1.03	same
SmolLM2-360M	16384	16	1,168	1,136	946	769	1.01	same

Table 7: llama.cpp serving concurrent clients (its parallel example: continuous batching, 8 requests per client, a shared system prompt, greedy decoding), under glibc and under `libmpr` with reservations from `libmpr`'s own profile of a smaller load (2 clients, 8 requests). Peak RSS and commit, mean of 3 runs; Time: `libmpr`'s run time over glibc's; Outputs: every request's prompt and response token counts, the same in all six runs.

Input	Preloaded (GCC build)			Built with `MemPrintResize` (clang)		
	none	own	held out	none	own	held out
135M, 32 tokens, commit	404.5	55.6	56.6	405.0	56.0	57.1
135M, 32 tokens, RSS	380.0	175.1	175.2	382.4	177.5	177.7
135M, 128 tokens, commit	404.5	56.4	56.2	405.0	56.9	56.6
135M, 128 tokens, RSS	380.1	177.4	177.5	381.8	179.6	179.9
360M, 64 tokens, commit	560.5	56.3	57.4	560.9	56.9	57.9
360M, 64 tokens, RSS	768.2	419.0	424.4	770.0	421.5	427.6

Table 8: llama.cpp, peak commit and peak RSS in MB: the GCC build with `libmpr` preloaded (sites are return addresses) against the clang build with `MemPrintResize` (sites are the pass's identifiers), each with no reservations, its own profile and held-out profiles.

and the interpreter's sites were keyed by that build's site map. Every site of 64 KB or more in Pin's profile was found by the interpreter under the same identifier, and no other; the 10 to 13 sites per run are mostly `pvector` buffers, whose allocation is inlined into the kernels at different depths, and BFS's frontier queues. Requested bytes agree within 0.3%, total high water within 0.5%, and each site's high water within 0.1% of its size, except BFS's three frontier queues (up to 77% of a 64 KB queue at scale 18), whose use depends on the traversal. For those, the interpreter's high water was the larger one in every case, because it follows both sides of data-dependent branches, which errs in the safe direction for a reservation.

Only one site in these kernels requests much more than it uses: sssp's frontier, a `pvector` sized to the number of edges (`sssp.cc:92`), a block of 7.3 to 30.4 MB per trial of which 11 to 32 KB is touched. Fitted from three scales, the interpreter's profiles give it $0.000987\,n + 2.1$ KB and Pin's $0.00103\,n + 1.1$ KB, and both commit 12 to 32 KB of each frontier with no block used beyond its commitment. Fitted from scale 16 alone, both routes give a constant that larger scales outgrow by 7 to 20 KB per block. We ran all 12 kernel and scale pairs with the corpus's arguments (4 or 8 trials), without reservations and with the reservations of each route (3 runs each): every run

Measure	darknet	llama.cpp
Allocation sites rewritten	631	5,137
Zeroing sites, growth points, inlined capacity call sites, growth contexts	–	720, 442, 2,315, 2,066
Build CPU time without / with the pass (s)	24.59 / 24.58	738.2 / 758.6
Site ids unchanged after inserting lines (previous pass / `MemPrintResize`)	80.9% / 96.6%	–

Table 9: What the pass costs and how stable its identifiers are. darknet: mean of three builds; llama.cpp: one build each, while other jobs ran. The id test inserts three lines at the top of five darknet files and one line inside `make_connected_layer`; with `MemPrintResize`, the 22 identifiers that change are all allocations in that function below the inserted line.

23 programs, largest inputs	glibc	libmpr, own	libmpr, held out
Total peak commit (MB)	2,504	2,236	2,482
Total peak RSS (MB)	2,290	2,292	2,291
Median commit / touched	1.01	1.01	1.01
Median RSS / touched	1.05	1.06	1.06
Time relative to glibc (range)	1	0.95 to 1.07	0.86 to 1.07
Programs with identical output	–	23 of 23	23 of 23

Table 10: The 23 programs of the scheduling corpus at their largest inputs (3 runs each), with the refined system.

produced the same output, peak RSS and commit were the same with either route's reservations, and time stayed within 0.99 to 1.04 of the build without reservations. The frontier is not live at the peak, which comes while the graph is built, so it lowers the commit while sssp runs but not its peak (except at scale 16, 32.5 to 31.7 MB).

Cost. We timed both profilers again, one process at a time on the idle machine (Table 11). Pin's exact `-sites` profile ran 65 to 97 times slower than the program (15 to 82 s) and held 0.46 to 1.5 GB. The interpreter ran 86 to 1,344 times slower (19 s to 20 min) and held 1.4 to 11.4 GB, so it took 1.3 to 14.8 times as long as Pin and 3 to 7 times its memory; over all twelve profiles, 2,751 s against 472 s. The ratio depends on the kernel more than on the scale: cc 1.3 to 1.4, bfs 1.9 to 2.1, sssp 3.0 to 3.3 and pr 13 to 15. Both slowdowns fall slightly with scale, the interpreter's from 1,344 to 1,234 on pr and 286 to 223 on sssp, and its memory grows with the input, about in proportion to the program's (1.4 GB at scale 16 to 5.1 GB at 18 for pr, 3.2 to 11.4 GB for sssp). So it is not a cheaper way to profile a program that Pin can run. What it shows is that the per-site profile, and the reservations, can come from the source alone, with no instrumented run, and agree with the measured ones on the code it can execute. Its limit is the programs it can run: GAP's and the regular C/C++ codes of our earlier study [31], not miniVite (MPI), darknet or the LLM engines, which read their sizes from model files.

10.12 Threads' stacks

Table 12 shows the stacks on every corpus program and on llama.cpp, whose engines start four threads. miniVite's commit falls from 45.1 to 16.5 MB, because its four OpenMP threads no longer commit 8 MB each; llama.cpp's falls from 56.1 to 22.5 MB with the 135M model and from 56.8 to 23.3 MB with the 360M model, a further 60% below the reservations alone and 94 to 96% below glibc. Quicksilver and MiniFE lose 8.4 MB each. The single-threaded programs do not change,

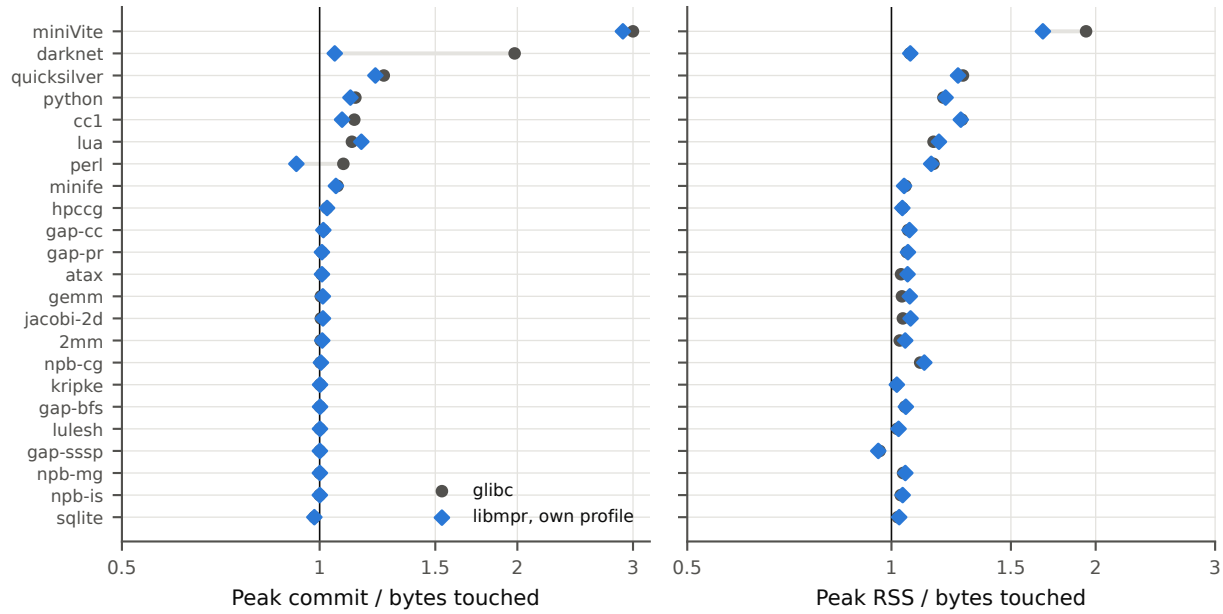


Figure 8: Peak commit and peak RSS relative to bytes touched for the 23 corpus programs, under glibc and under `libmpr` with each program's own profile. Most programs already sit at about 1.

RSS does not change, and every output is identical.

10.13 Profiles at native speed

Table 13 replaces the Pin profile by `libmpr`'s. Over the 28 configurations the reservations commit 2,391 MB against 2,383 MB from Pin's profiles and hold 3,578 MB of RSS against 3,581, every output is identical, and run time is 0.92 to 1.03 of glibc's. The profiles took 95 s in all, against 21,676 s under Pin, from 0.07 to 15 s each. The one program that differs is Perl (61.5 against 52.2 MB): its large buffers grow by `realloc`, and `libmpr` records each at its final size, which the program has used almost entirely, so the fit keeps no reservation for the site; from the Pin profile, which records every buffer of the chain, it kept one.

10.14 What a profile costs

A reservation is only as cheap as the profile behind it. Table 14 records what the Pin tool's exact `-sites` mode cost on every other workload: a median slowdown of 217 over 29 profiles, from 40 on llama2.c (single-threaded) to 452 to 596 on llama.cpp (four threads), and 0.3 to 4.2 GB of memory. The full trace is what makes the profile exact; whether a sampled or windowed `-sites` profile would give the same reservations is open (Section 12). Against GAP's 65 to 97 when each run was alone (Table 11), the corpus's 76 to 127 for GAP at scale 18 includes the contention of the profiles that ran beside it and four to eight trials. Because the savings held with one profile per model (Section 10.7), the cost is paid once per program and model, not per run, and the interpreter can replace it only for the programs it can execute, at a further 1.3 to 15 times the time (Section 10.11).

			High water (MB)		Largest	Native	Pin `-sites`		Interpreter		Peak RSS (MB)
Kernel	Scale	Sites	static	Pin	site diff.	(s)	(s)	slowdown	(s)	slowdown	Pin / interp.
pr	16	11 / 11	27.5	27.5	0.1%	0.19	19	97×	260	1,344×	459 / 1,425
pr	17	11 / 11	55.3	55.3	0.1%	0.43	40	94×	527	1,236×	726 / 2,649
pr	18	11 / 11	111.2	111.2	0.0%	0.99	82	83×	1217	1,234×	1,254 / 5,069
bfs	16	13 / 13	27.3	27.2	22.9%	0.18	15	81×	31	171×	457 / 1,358
bfs	17	13 / 13	54.8	54.9	21.0%	0.38	27	72×	52	137×	723 / 2,573
bfs	18	13 / 13	110.4	110.2	77.3%	0.84	55	65×	113	134×	1,253 / 4,850
cc	16	10 / 10	27.2	27.2	0.1%	0.17	15	85×	19	108×	458 / 1,389
cc	17	10 / 10	54.8	54.8	0.1%	0.38	27	71×	37	97×	725 / 2,574
cc	18	10 / 10	110.1	110.1	0.0%	0.83	54	65×	72	86×	1,253 / 4,851
sssp	16	11 / 11	47.1	47.1	0.1%	0.22	20	88×	64	286×	534 / 3,230
sssp	17	11 / 11	94.8	94.9	0.1%	0.50	39	78×	123	246×	871 / 6,263
sssp	18	11 / 11	190.9	190.9	0.0%	1.06	80	75×	237	223×	1,542 / 11,416
Total						6.2	472		2,751		

Table 11: The interpreter's per-site profiles (Section 9) against the Pin tool's, and what each cost, for GAP's kernels built with `MemPrintResize` (Kronecker graphs, one trial), blocks of at least 64 KB. Sites: in both profiles under the pass's identifiers, of those in Pin's. Largest site difference: the largest difference in a site's high water, as a share of its requested bytes. Times are wall times with one process on the otherwise idle machine (native: mean of 3 runs of the same build); slowdown is over native; peak RSS of the profiling process (Pin with the program, or the interpreter's Python process).

Program	glibc commit	stacks off	stacks on	RSS off / on	Time on
llamacpp-135m-n32	404.6	56.1	22.5	176.2 / 176.3	0.91
llamacpp-360m-n64	560.4	56.8	23.3	419.9 / 420.0	0.87
miniVite	46.2	45.1	16.5	24.2 / 25.8	1.01
minife	273.0	272.0	263.6	267.8 / 267.8	0.98
quicksilver	52.8	51.8	43.4	52.9 / 53.0	1.00
20 other programs	2,132	1,878	1,877	1,946 / 1,942	0.95–1.13

Table 12: `libmpr`'s `MAP_NORESERVE` stacks (Section 8.7) on the corpus's largest inputs and llama.cpp, with each program's own reservations: peak commit and RSS in MB, time relative to glibc, mean of 3 runs. The programs whose commit changes by more than 0.5 MB are listed; every output is identical.

10.15 Robustness to wrong predictions

Under-prediction is safe by construction, and we tested it. With every prediction multiplied by 0.1 (`MPR_SCALE=0.1`), darknet and miniVite produced identical outputs, and the telemetry reported the blocks that used more than predicted. The quantile in the fit leaves blocks uncovered by design, and held-out profiles under-predict on the LLM engines and the corpus, again with identical outputs. Over-prediction costs commit and nothing else, because the committed part is still backed lazily. A container capacity can only raise the capacity a container would have chosen. Two control programs with no waste, PolyBench 2mm and HPCCG, with `libmpr` serving every large block (`MPR_ALL`), changed by at most 1.8 MB of RSS and 0.1 MB of commit.

10.16 Effect on scheduling

Our earlier study replayed the corpus's measured footprints through a simulator of 32 nodes with a shared memory pool [32]. We added a third truth, the RSS of each program under `libmpr` (Table 15). The results hardly move from the RSS truth, because the corpus programs' resident

memory hardly moved: the exact curve gives 103 minutes (91 under the RSS truth) and MemPrint's windowed curves 1,727 (1,807). The refined system leaves the corpus's total RSS unchanged as well. The scheduling study's conclusions therefore do not depend on resizing allocations, since its gain comes from reserving a curve instead of a peak. That study scales every job so that its peak is the stream's, under each truth, so a job holds as much under `libmpr` as under glibc there; Section 10.18 keeps one scale per job and measures what the memory `libmpr` saves buys a cluster. Resizing matters for programs that commit or hold much more than they touch, such as darknet and the LLM engines, and for systems with strict overcommit, where the saved commit is memory that can be promised to other jobs.

10.17 Against Pond, dynamic provisioning and KS+

Three published systems reduce the memory a job is given: Pond [20] places the part of a virtual machine's memory it predicts will stay untouched in a CXL pool, the dynamic provisioning of Zacarias et al. [48] resizes a job's allocation to its use every five minutes, and KS+ [4] reserves a step function over time that it learns from earlier executions with other inputs. They act on what a system holds for the job; `MemPrintResize` and `libmpr` act on what the job commits and touches. We compare them on the recorded runs of this section (`paper_resize/baselines.py`), with the policies of our scheduling study [32], where all three are implemented as scheduler oracles and checked against the authors' code where it is published. Every approach starts from an exact request, the job's peak of commit and RSS under glibc, so none gains from a user's overestimate. Pond holds the request and keeps in local DRAM only the request minus its predicted untouched memory, taken from the job's other runs of the same input (its per-customer history at its best). Dynamic provisioning holds the request for the first sixth of the run (five minutes of a median 30) and then each sixth's largest use, read from the run itself, as their simulator reads it from the usage trace. KS+ predicts from the RSS of the program's other corpus inputs and re-runs the job with its retry strategy when the use exceeds the reservation; we report it with the authors' 100 MB floor on every predicted peak and with a 1 MB floor, since the 100 MB floor exceeds the whole footprint of many of our programs. `MemPrintResize` is given the same exact request under `libmpr`: the peak of commit and RSS of the run with its reservations.

Table 16 gives the totals and Table 17 the workloads. `libmpr` with its own profile holds 3,765 MB on time average against 4,905 MB for the request, 23% less, and 18% less with a held-out profile, under either demand. Dynamic provisioning holds 10% less than the request when the demand is the RSS and 4% less when it includes the commit, which it cannot change; with a resize every sixtieth of the run it holds 3,763 MB under the RSS demand, as much as `libmpr`, but only because each resize is the coming interval's largest use, which a running system cannot know. Pond holds the whole request, as it never fails, and keeps 6% of it out of local DRAM, almost all of it darknet's dead training buffers (247 of 299 MB), which `libmpr` does not commit at all. KS+ holds more than the request: its extrapolation from the two smaller inputs reserves 8.5 times GCC's cc1 peak (1,140 against 134 MB), and with the authors' floor every small program holds at least 100 MB. Without cc1 and with a 1 MB floor it holds 2% less than the request, with 36 failures over all runs.

The gain of `libmpr` that none of the three can obtain is on the LLM engines. Under glibc, llama.cpp touches its whole KV cache when it clears it, so its RSS is its commit, and no system can give it less without killing it; with `libmpr`, the clearing skips untouched pages, and the RSS itself falls from 380 to 177 MB (135M model) and from 767 to 419 MB (360M model). The approaches compose: dynamic provisioning applied to the runs under `libmpr` holds 3,553 MB, 28% less than the request, and Pond's local DRAM under `libmpr` is 3,763 MB, because `libmpr` has already removed most of the untouched memory Pond would place in the pool.

10.18 Cluster throughput

The comparison of Section 10.17 looks at one job at a time. To measure what each approach is worth to a cluster, we replay the 29 workloads through DMSched-Sim [32], with Pond, dynamic provisioning and KS+ as its scheduler policies (`analysis/memprint/sched/resize_cluster.py`). The stream is the scheduling study's synthetic one: 3,000 jobs with 1 to 16 cores, a median runtime of 30 minutes and a median request of 100 GB. Each job runs one of the 29 workloads, chosen at random, with the RSS and commit timelines recorded for it under glibc and under `libmpr` (own profile), and it requests what its program allocates, its peak of commit and RSS. One scale per job maps the glibc allocation to the stream's request, so a job built with the pass is smaller in the cluster by exactly what `libmpr` saves; both builds face the same arrivals, and the offered load is the glibc stream's. A job is OOM-killed when its demand (its RSS, or the larger of its commit and RSS) exceeds what it is given. Under glibc we run the request, Pond, dynamic provisioning and KS+; with the pass, the request alone (`MemPrintResize` by itself), and with dynamic provisioning or Pond on top. In "`MemPrintResize` + dynamic", the job is built with the pass, so it commits and touches less, and the scheduler applies dynamic provisioning to it: the job starts with its smaller request, and every five minutes its allocation is resized to the largest use of the next five minutes of that smaller footprint. The two savings add: the program holds less, and the scheduler does not hold memory the program has not reached yet. The exact curve of each build bounds what any policy can reach.

`MemPrintResize` is the only one of the approaches that raises the throughput (Figure 9, Table 18). From load 0.6, where the cluster is saturated, the request under glibc completes 21.4 to 21.6 jobs per hour and `MemPrintResize` 25.0 to 25.3, 16 to 17% more, under either demand. Pond, dynamic provisioning and KS+ stay within 21.5 jobs per hour or fall below it: Pond holds the whole request, dynamic provisioning restarts 650 to 830 jobs per stream whose growth no longer fits, and KS+'s about 6,300 OOM kills (7,250 when the demand includes the commit) waste the work they interrupt. They change how the same demand is held; `MemPrintResize` makes the jobs smaller. Dynamic provisioning does shorten the mean response time, from 1,827 to 1,136 minutes at load 0.8 under the RSS demand, by starting jobs earlier on memory it later reclaims, against 1,225 for `MemPrintResize`. The two compose: dynamic provisioning on the jobs built with the pass gives 718 minutes and 23.8 jobs per hour, the shortest response time of any policy short of the exact curves, and needs 5.0 TB for the target service, against 6.0 TB for the request and for dynamic provisioning alone and 7.0 TB for KS+. With the exact curve, the glibc build completes 30.5 jobs per hour at load 0.8 and the build with the pass 34.6, 13% more: what `libmpr` saves adds to what a perfect footprint oracle gains.

Over-requests. Every job above requests exactly what its program allocates, which is the case most favourable to the request. Users ask for more, and Zacarias et al. study a 60% overestimate [48]. Table 19 raises every glibc-build request by 25, 60 and 100%. The request falls from 21.6 to 14.5, 9.4 and 6.8 jobs per hour and Pond, which holds the request, from 21.2 to 6.0. Dynamic provisioning holds the inflated request only for its first interval and falls less, to 19.0 at +60% and 15.4 at +100%. KS+ reserves the request only for darknet, which has no other input, and falls from 21.5 to 18.9, because darknet's jobs are among the largest. `MemPrintResize` takes its reservations from the profile, not from what the user writes, so it stays at 25.0, 3.7 times the request at +100%. At +60% the request needs 11.0 TB for the target service and Pond does not meet it at 320 GB per node, while `MemPrintResize` needs 6.0 TB and `MemPrintResize` with dynamic provisioning 5.0 TB.

A running monitor. Dynamic provisioning as its authors simulate it reads the coming five minutes of each job's use. A running system knows only the past, so we also run a *look-back* variant: each update sets the allocation to the previous five minutes' largest use, the job grows on demand whenever its use exceeds it, memory is committed a minute at a time, and after three restarts a job runs with its request held, as Zacarias et al. suggest for jobs that fail repeatedly; without that, a job whose growth the scheduler keeps giving to newer jobs restarts indefinitely. It gives 21.1 jobs per hour and 1,137 minutes at load 0.8, with 1,118 restarts and 117 fallbacks per stream, as the look-ahead version does (21.2, 1,136): committing a minute at a time instead of the coming interval's peak makes up for not knowing the future. With its costs, its growth on demand adds 1,098 core-hours, against 396 for the look-ahead version. On the jobs built with the pass, the look-back variant gives the shortest response time of any policy short of the exact curves, 665 minutes with 24.6 jobs per hour.

Held-out profiles. The builds above use each workload's own profile. With reservations fitted on the program's other inputs or models (Section 10.7), `MemPrintResize` completes 24.3 jobs per hour at load 0.8 against 25.0 with its own profile and 21.6 for the request, and with dynamic provisioning 24.1 jobs per hour and 797 minutes. A profile from other inputs keeps most of the gain.

10.19 What each policy costs

The results above give every policy its decisions for free. Each one has a cost of its own, which we add to the simulation (`-overheads`) from what was measured in this study or reported by the systems' authors:

- `MemPrintResize` runs every job built with the pass at its measured `libmpr` runtime ratio (median 0.996, 0.93 to 1.07 across the 29 workloads), and it needs a profile: the first job of each workload in the stream is its profiling run, under `libmpr`'s native-speed profiler (Section 8.8; 0.97 times the glibc run at the median, at most 1.17) or the Pin tool (219 times at the median), holding the glibc build's request.
- **Dynamic provisioning** loses 10 s of progress for every resize that grows a job, as in the authors' simulator, and its restarts lose the work done before them.
- **Pond**'s mitigation copies the job's pool memory to local memory at 50 ms per GB, as the Pond paper reports for its one-time correction.
- **KS+** loses the work of every attempt its retry strategy replaces. Its model needs only the monitored runs of earlier jobs and a regression per segment, which we count as free, as we do Pond's telemetry and training.

Table 20 gives the result at load 0.8. Pond's 27 mitigations per stream cost under a core-hour. Dynamic provisioning resizes jobs about 18,000 times per stream; the growing ones add 395 core-hours, and with its 842 restarts it falls from 21.2 to 20.6 jobs per hour and its response time rises from 1,136 to 1,271 minutes. KS+ loses 8,322 core-hours to its retries. `MemPrintResize` with the native-speed profiler loses nothing: its 29 profiling runs take 198 core-hours, its runtime ratio is one at the median, and it stays at 25.1 jobs per hour. With the Pin tool, the same 29 profiling runs take 47,539 core-hours, since a 30-minute job takes about four and a half days to profile; within one stream of 3,000 jobs they dominate the cluster, the throughput falls to 3.1 jobs per hour, and the memory needed for the target service rises from 6.0 to 9.0 TB. A Pin profile is therefore an

offline cost, to be paid once and reused over many more runs than one stream holds, while the native-speed profile can be taken on the job's own first run. With dynamic provisioning on top, `MemPrintResize` keeps the shortest response time of any policy (803 minutes with all costs) and needs 5.0 TB for the target service, against 6.0 TB for the request.

11 Discussion

11.1 The time axis of a footprint curve

In the scheduling study, a MemPrint curve needed about 60% headroom to cover the RSS truth, and we attributed part of that to the time axis, since MemPrint counts memory references while RSS is sampled in seconds [32]. The new options let us test the attribution. We reran the full trace and the 5% windowed mode at every program's largest input with `-footprint pages`, and mapped each curve to the wall time recorded in the `Seconds` column (Table 21, Figure 10). On wall time, the full trace's 90th-percentile margin falls from 0.58 to 0.44, and the mean response time from 1,847 to 1,476 minutes. The programs that gain most have phases that are short in references but long in time, such as GAP BFS (margin 0.53 to 0.06), GAP CC (0.56 to 0.09) and Lua (0.70 to 0.05). Counting pages instead of bytes helps the programs whose touched bytes are scattered (cc1 0.25 to 0.13, Perl 0.29 to 0.13, miniVite 1.12 to 0.84), but it does not reduce the calibrated margin on its own. Together the two options give 0.41 and 1,423 minutes, still far from the exact peak's 436 minutes. The remaining margin comes from memory that MemPrint does not count, such as miniVite's library code, which the scheduler's RSS baseline (the run's smallest sample, taken before MPI loads its libraries) treats as footprint. The wall time is measured under Pin, which slows phases unevenly, so it approximates native time.

11.2 Bytes touched can exceed the resident set

With the first profile, llama.cpp's RSS under `libmpr` (193 MB) fell below its bytes touched as MemPrint measured them (353 MB at the peak, with the mapped weights). The difference was the zeroing `memset`: writing zeros to fresh anonymous memory counts as touching it, although no reader can distinguish the result from untouched memory. `-skip_zeroing` now leaves such writes out of the profile (Section 5), which is what let `libmpr` commit 1 MB of the KV cache instead of 189 MB. It is on with `-sites` only: the footprint truth of the scheduling study keeps its definition, in which a write is a touch whatever it writes. Whether the truth should also leave out zeroing of untouched memory is a question of definition. Under it, the truth would follow the resident set of a program run with `libmpr` more closely, and the resident set of a program run without it less closely.

11.3 What the RSS numbers measure

The RSS figures in this paper are peaks of samples of `smaps_rollup`, taken every 20 ms, so a spike shorter than the interval can be missed. They include file-backed pages and count a huge page as 2 MB. The per-buffer figures for llama.cpp come from a different instrument. A logging library recorded where glibc (or `libmpr`) placed each buffer of at least 1 MB, without changing how it allocated, and at the sample with the largest total RSS we counted each buffer's pages that pagemap reports as present and mapped exclusively, which excludes the shared zero page. Under glibc the KV cache and the compute buffer lie in one merged mapping, so `smaps` alone cannot separate them.

11.4 Where resizing pays

The results separate three cases. A program that allocates buffers it never touches (darknet, llama.cpp's scheduler arrays) gains commit but no RSS, because the kernel never backed those buffers. A program whose allocator or own code touches memory it does not need (a zeroing `memset`, vector doubling, huge pages over sparse writes) gains RSS, and only a change in how the memory is obtained recovers it. A program that already uses what it allocates, which is the median program in our corpus, gains nothing and loses nothing. Policies that act on what a system holds, such as Pond, dynamic provisioning and KS+, can recover the first case's commit only by reserving less than the request, and none of them the second case's RSS; in a cluster this is the difference between holding the same jobs differently and running smaller jobs (Section 10.18).

12 Limitations and future work

A limitation of the container capacities is that a capacity measured on one input transfers to another only as far as the final sizes agree. The final size of miniVite's edge list depends on the graph, so a capacity from the 8192- and 16384-vertex inputs reserves the 16384-vertex size, half of what the 32768-vertex input needs, and the vector still grows once more (Section 10.5). Predicting the final size from the input would need a model of size against the input's parameters, as the `periodic` model does for KV caches, or a per-site profile from source (Section 9), which the interpreter cannot produce for miniVite, an MPI program. The growth points are specific to the containers of libstdc++ and libc++ listed in Table 2; `std::deque` grows in fixed 512-byte nodes and has no capacity to raise.

A limitation of the shaped reservations is that a range is a fraction of the block, so it transfers between blocks of the same layout and use. llama2.c's held-out reservations, from models with 6 and 8 layers, cover a 12-layer model's KV caches only partly, and commit 46 MB against 12 MB with the model's own profile. The `periodic` model names the layout and follows the run's length (Section 10.7), but its number of periods and bytes per token are a model's, so it is fitted per model, and its parameter has to be supplied by whoever starts the job; without it, `libmpr` assumes the largest value profiled. A parameter is only as good as its relation to the use: llama2.c's steps bound its tokens from above, so a run that ends early is over-committed. The `ranges` model's starts are fractions of the block, so they hold for one layout: llama.cpp's compute buffer at another context length is laid out differently (Section 10.7).

Calling contexts are taken with a backtrace for each large allocation at a site that has context lines. The frames are return addresses, so they are tied to a build, as image keys are, and a context deeper than two frames is not tried.

Small blocks are left to glibc, so the per-chunk overhead of small allocations, 1.42× in miniVite, is not addressed. Allocators inside interpreters (pymalloc, Lua's tables) and custom `operator new` are not covered beyond the large blocks they request from the system.

The commit saving depends on the overcommit policy. Under strict overcommit (`vm.overcommit_memory=2`) the kernel ignores `MAP_NORESERVE`, so the saving disappears, although correctness is unaffected; we could not set the policy on our machine to measure it. The zero skipping reads pagemap, which unprivileged processes may read on this kernel without page frame numbers, and its zero-page check treats a page shared after `fork` conservatively by reading it; no workload we ran forks. Keeping huge pages off the skipped pages cost no time on llama.cpp, but a program that clears a buffer and then fills all of it densely would run on 4 KB pages there.

Profiling under Pin is expensive: the Pin tool's exact mode with `-sites` took 300 to 1,100 s for llama.cpp runs of 0.8 to 2 s, and 617 s for llama2.c 110M (18 s natively). `libmpr`'s own profile

removes that cost for the large blocks a reservation needs, at the price of whole pages and final sizes (Section 8.8); the Pin tool is still needed for container growth chains, small blocks and byte-exact footprints. The interpreter avoids the instrumented run but was slower than Pin on GAP, 1.3 to 14.8 times its time and 3 to 7 times its memory (Section 10.11, Section 10.14). An LLM server that runs many sequences fills its KV cache over time, and its RSS savings shrink as it does, to 3 to 4% with 16 concurrent clients, while its commit savings remain (Section 10.8); we measured a server's load from llama.cpp's own client simulation, not from a real request trace.

The comparison with the published policies has its own limits. The cluster's job stream is synthetic, with each job one of 29 workloads scaled from tens or hundreds of MB to tens of GB, which assumes the savings scale with the job. Dynamic provisioning reads the coming five minutes of a job's use from the recorded run, as its authors' simulator does, which favours it. Pond's policies are not in its artifact and follow its paper. KS+ learns from two smaller inputs per program and with the authors' 100 MB floor, which suits GB-scale tasks better than ours. The profiling cost assumes one profile per workload in a stream of 3,000 jobs; a site that runs a program many more times amortizes even a Pin profile.

13 Conclusion

We passed MemPrint's measured per-site footprints to a compiler pass and a runtime as non-binding reservations, so that a program is backed by what it touches rather than what it requests, and an under-prediction costs accounting instead of correctness. Every program we ran produced its original output. Calling contexts, shaped reservations and a profile that does not count zeroing as use took the system from a 34% to an 86% commit reduction on llama.cpp and from 7% to 85% on llama2.c, and zero skipping halves llama.cpp's resident memory. Per-call-site container capacities, with element sizes from the debug information, remove miniVite's vector growth without slowing it, once the runtime's lookup is fast enough for a million growths. The 23-program corpus, which already holds about what it touches, saw no regressions. The measurements also showed that a footprint curve is most useful to a scheduler on a wall-clock axis, and that what a profile counts as use, such as zeros written to fresh memory, decides what can be saved. A profile describes its input: on unseen LLM inputs the savings held, and a reservation that takes the KV cache's layout from the profile and its length from the run kept the commitment in step with the use. The per-site profile can also come from source: MemPrint's interpreter reproduces Pin's profiles of GAP site by site, so the static route can now feed the same reservations, for the programs it can run.

The runtime can do more of the work itself. Taking the profile from the pages each block touched, in one native run, gave the same reservations as the Pin profile at a two-hundredth of its cost; mapping threads' stacks `MAP_NORESERVE` removed most of the commit that remained in the multithreaded programs; and a second run parameter, the prompt's length, let the reservations follow llama.cpp's compute buffer as the first follows its KV cache.

Compared with policies that act on what a system holds for a job, such as Pond, dynamic provisioning and KS+, changing what the job itself commits and touches is what raises a cluster's throughput: in our simulation `libmpr` completes 16% more jobs per hour than the request, where the three published policies complete none more, and dynamic provisioning applied to jobs built with the pass shortens the response time further. The profile that makes this possible must be cheap where it is taken: `libmpr`'s native-speed profile costs one run at native speed, while a Pin profile belongs offline.

References

[1] Loubna Ben Allal et al. SmolLM2: When smol goes big – data-centric training of a small language model. arXiv:2502.02737, 2025.

[2] Glenn Ammons, Thomas Ball, and James R. Larus. Exploiting hardware performance counters with flow and context sensitive profiling. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 85–96, 1997.

[3] Amir Ayupov, Maksim Panchenko, and Sergey Pupyrev. Stale profile matching. In *Proceedings of the 33rd ACM SIGPLAN International Conference on Compiler Construction (CC)*, pages 162–173, 2024.

[4] Jonathan Bader, Ansgar Lößer, Lauritz Thamsen, Björn Scheuermann, and Odej Kao. KS+: Predicting workflow task memory usage over time. In *IEEE 20th International Conference on e-Science*, 2024.

[5] Michael D. Bond and Kathryn S. McKinley. Probabilistic calling context. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, pages 97–112, 2007.

[6] Dehao Chen, David Xinliang Li, and Tipp Moseley. AutoFDO: Automatic feedback-directed optimization for warehouse-scale applications. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, pages 12–23, 2016.

[7] DWARF Debugging Information Format Committee. *DWARF Debugging Information Format, Version 5*, 2017.

[8] Jason Evans. A scalable concurrent malloc(3) implementation for FreeBSD. In *BSDCan*, 2006.

[9] Free Software Foundation. *The GNU C Library Reference Manual: Source Fortification.* `https://sourceware.org/glibc/manual/`.

[10] Free Software Foundation. *The GNU Linker (ld):* `-build-id`. GNU Binutils documentation.

[11] GCC. *The GNU C++ Library Manual, Chapter 19: Profile Mode.* `https://gcc.gnu.org/onlinedocs/gcc-4.7.1/libstdc++/manual/manual/bk01pt03ch19s07.html`.

[12] Georgi Gerganov and ggml contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, release b11500, 2026.

[13] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, and Assefaw H. Gebremedhin. miniVite: A graph analytics benchmarking tool for massively parallel systems. In *Proceedings of the IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems Workshop (PMBS)*, 2018.

[14] A. H. Hunter, Chris Kennelly, Paul Turner, Darryl Gove, Tipp Moseley, and Parthasarathy Ranganathan. Beyond malloc efficiency to fleet efficiency: A hugepage-aware memory allocator. In *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 257–273, 2021.

[15] Changhee Jung, Silvius Rus, Brian P. Railing, Nathan Clark, and Santosh Pande. Brainy: Effective selection of data structures. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 86–97, 2011. Also in ACM SIGPLAN Notices 46(6).

[16] Andrej Karpathy. llama2.c: Inference Llama 2 in one file of pure C. `https://github.com/karpathy/llama2.c`, 2023.

[17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[18] Youngjin Kwon, Hangchen Yu, Simon Peter, Christopher J. Rossbach, and Emmett Witchel. Coordinated and efficient huge page management with Ingens. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 705–721, 2016.

[19] Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis and transformation. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, 2004.

[20] Huaicheng Li et al. Pond: CXL-based memory pooling systems for cloud platforms. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[21] The Linux Kernel Documentation. *Examining Process Page Tables*. Documentation/admin-guide/mm/pagemap.rst.

[22] The Linux Kernel Documentation. *Overcommit Accounting*. Documentation/mm/overcommit-accounting.rst.

[23] The Linux Kernel Documentation. *Transparent Hugepage Support*. Documentation/admin-guide/mm/transhuge.rst.

[24] Lixia Liu and Silvius Rus. Perflint: A context sensitive performance advisor for C++ programs. In *Proceedings of the 7th IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 265–274, 2009.

[25] The LLVM Project. *Clang Compiler User's Manual: Sample Profile Text Format*. `https://clang.llvm.org/docs/UsersManual.html`.

[26] The LLVM Project. *MemProf: Memory Profiling for LLVM*, 2025. `https://llvm.org/docs/MemProf.html`.

[27] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '05, page 190–200, New York, NY, USA, 2005. Association for Computing Machinery.

[28] Martin Maas, David G. Andersen, Michael Isard, Mohammad Mahdi Javanmard, Kathryn S. McKinley, and Colin Raffel. Learning-based memory allocation for C++ server workloads. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.

[29] Lukas Makor, Sebastian Kloibhofer, Peter Hofer, David Leopoldseder, and Hanspeter Mössenböck. Automated profile-guided replacement of data structures to reduce memory allocation. *The Art, Science, and Engineering of Programming*, 10(1), 2025.

[30] Microsoft. *VirtualAlloc function (memoryapi.h)*. Win32 API documentation, `https://learn.microsoft.com/windows/win32/api/memoryapi/nf-memoryapi-virtualalloc`.

[31] Nafis Mustakin. From source code to sampling correction: Predicting memory-footprint models for unseen programs. Manuscript, 2026.

[32] Nafis Mustakin. What is an exact memory footprint worth to a scheduler? measured footprints and job scheduling on disaggregated memory. MemPrint repository, paper_sched, 2026.

[33] Nafis Mustakin, Joseph Manzano, and Daniel Wong. MemPrint: Constructing program memory footprint estimations using statistical methods from sparsely sampled Pin-based memory traces. Manuscript, 2025.

[34] Rei Odaira. Information processing apparatus and program and method for adjusting initial array size. U.S. Patent 9,218,191 B2, assigned to International Business Machines Corp., 2015.

[35] Oracle. *Linker and Libraries Guide: Using Interposition*. Oracle Solaris 11.4 documentation.

[36] Ashish Panwar, Sorav Bansal, and K. Gopinath. HawkEye: Efficient fine-grained OS support for huge pages. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.

[37] Bobby Powers, David Tench, Emery D. Berger, and Andrew McGregor. Mesh: Compacting memory management for C/C++ applications. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2019.

[38] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. vAttention: Dynamic memory management for serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.

[39] Joseph Redmon. Darknet: Open source neural networks in C. `https://pjreddie.com/darknet/`, 2016.

[40] Niklas Röjemo and Colin Runciman. Lag, drag, void and use: Heap profiling and space-efficient compilation revisited. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 1996.

[41] Joe Savage and Timothy M. Jones. HALO: Post-link heap-layout optimisation. In *Proceedings of the 18th ACM/IEEE International Symposium on Code Generation and Optimization (CGO)*, 2020.

[42] Ohad Shacham, Martin Vechev, and Eran Yahav. Chameleon: Adaptive selection of collections. In *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2009.

[43] Valgrind Developers. *DHAT: a dynamic heap analysis tool*. Available at `https://valgrind.org/docs/manual/dh-manual.html`.

[44] Guoqing Xu, Nick Mitchell, Matthew Arnold, Atanas Rountev, Edith Schonberg, and Gary Sevitsky. Finding low-utility data structures. In *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 174–186, 2010.

[45] Guoqing Xu and Atanas Rountev. Detecting inefficiently-used containers to avoid bloat. In *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 160–173, 2010.

[46] Xi Yang, Stephen M. Blackburn, Daniel Frampton, Jennifer B. Sartor, and Kathryn S. McKinley. Why nothing matters: The impact of zeroing. In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 307–324, 2011.

[47] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Memory demands in disaggregated HPC: How accurate do we need to be? In *International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2021.

[48] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Dynamic memory provisioning on disaggregated HPC systems. In *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC-W)*, 2023.

	Profiling time (s)		Peak commit (MB)			Peak RSS (MB)
Program	Pin	libmpr	glibc	Pin's profile	libmpr's	Pin's / libmpr's
2mm	2,829	8.4	37.3	38.0	38.0	39.0 / 39.1
atax	16	0.1	32.3	32.8	32.8	33.9 / 33.9
cc1	397	1.7	119.2	114.6	114.5	134.0 / 135.3
darknet	968	2.8	534.7	285.1	285.1	287.8 / 287.7
gap-bfs	117	1.4	72.9	73.4	73.4	76.5 / 76.6
gap-cc	144	1.5	72.9	73.4	73.4	76.5 / 76.5
gap-pr	280	2.0	72.9	73.4	73.4	76.5 / 76.5
gap-sssp	204	1.8	135.9	136.4	136.4	126.9 / 126.9
gemm	1,948	5.0	29.2	29.9	29.9	31.0 / 31.0
hpccg	523	1.6	105.3	105.9	105.9	106.6 / 106.3
jacobi-2d	3,332	11.3	27.3	28.0	28.0	29.1 / 29.0
kripke	752	3.0	274.9	275.5	275.5	279.6 / 279.6
llama2c-110M	617	14.8	78.2	12.6	12.6	452.5 / 452.5
llama2c-15M	94	2.0	6.2	5.1	4.7	67.4 / 67.4
llama2c-42M	279	5.8	36.2	7.6	7.5	175.9 / 175.9
llamacpp-135m-n32	332	0.6	404.6	56.1	55.6	176.3 / 174.9
llamacpp-360m-n64	1,062	1.7	560.4	56.8	56.7	420.1 / 419.1
lua	458	2.6	63.9	66.5	66.5	68.2 / 66.8
lulesh	2,195	5.9	151.2	151.8	151.8	154.8 / 154.8
miniVite	559	1.6	46.2	45.1	45.1	25.8 / 25.7
minife	1,544	6.3	273.0	272.0	272.0	268.5 / 267.8
npb-cg	457	1.2	27.5	28.0	28.0	30.6 / 30.5
npb-is	188	1.8	69.7	70.2	70.2	72.4 / 72.5
npb-mg	59	0.3	58.5	59.1	59.1	61.3 / 61.2
perl	257	1.4	61.0	52.2	61.5	64.3 / 65.2
python	1,085	3.6	108.7	107.3	107.4	115.3 / 115.4
quicksilver	757	3.7	52.8	51.8	51.8	53.0 / 53.0
sqlite	225	1.0	76.4	74.5	74.3	77.5 / 77.3
Total	21,676	95	3,589	2,383	2,391	3,581 / 3,578

Table 13: Reservations from `libmpr`'s own profile (Section 8.8, one native run) against those from the Pin tool's `-sites` profile of the same input, on the corpus's largest inputs and the LLM engines: profiling time (Pin's from Table 14, upper bounds), and peak commit and RSS with each set of reservations, mean of 3 runs (thread stacks off, to compare the reservations alone).

Workloads	Native (s)	Pin `-sites` (s)	Slowdown	Peak RSS under Pin (GB)
PolyBench (2mm, gemm, jacobi-2d, atax)	0.1 to 12.4	16 to 3,332	121 to 346	0.3 to 0.5
GAP (bfs, pr, cc, sssp; scale 18)	1.5 to 2.2	117 to 280	76 to 127	1.2 to 2.0
HPC proxies (HPCCG, LULESH, miniFE, Kripke, Quicksilver)	1.7 to 7.0	523 to 2,195	183 to 396	0.9 to 3.3
NPB (IS, CG, MG)	0.4 to 2.0	59 to 457	92 to 273	0.5 to 1.2
Interpreters, database, compiler (Lua, Python, Perl, SQLite, cc1)	1.2 to 4.0	225 to 1,085	130 to 275	1.1 to 3.0
miniVite 32768	1.9	559	296	0.6
darknet AlexNet	3.0	968	325	4.1
llama2.c (15M, 42M, 110M; 128 steps)	2.1 to 15.3	94 to 617	40 to 48	0.5 to 2.6
llama.cpp (135M 32/128 tokens, 360M 64)	0.7 to 1.9	332 to 1,062	452 to 596	2.5 to 3.9
All 29			median 217	

Table 14: The Pin tool's exact `-sites` profile on the workloads the interpreter cannot run or that we profiled only under Pin: the corpus's largest inputs and the LLM profiles, against the same input's native run under glibc. The corpus profiles ran several at a time, so their times are upper bounds.

Oracle	Truth: bytes touched	Truth: RSS	Truth: RSS under libmpr
Exact curve	90	91	103
Exact peak	460	436	457
MemPrint W5	391	1,807	1,727
DHAT peak	979	1,277	1,230
User request	2,215	2,197	2,204

Table 15: Mean response time (minutes) at 80% offered load in the DMSched-Sim scheduler of our earlier study [32], with calibrated margins, under three truths of what a job holds. The third truth is each program's RSS when it runs with `libmpr` (first version) and its own profile.

	Demand = RSS		Demand = max(commit, RSS)	
Approach	Peak	Mean	Peak	Mean
Exact request (glibc)	4,912	4,905	4,912	4,905
Pond: all held	4,912	4,905	4,912	4,905
Pond: local DRAM	4,606	4,606	4,606	4,606
Dynamic provisioning	4,912	4,418	4,912	4,685
KS+ (100 MB floor)	6,956	6,578	6,956	6,610
KS+ (1 MB floor)	6,378	5,791	6,385	5,929
MemPrintResize	3,770	3,765	3,770	3,765
MemPrintResize, held-out profile	4,031	4,025	4,031	4,025
MemPrintResize + Pond: local	3,763	3,763	3,763	3,763
MemPrintResize + dynamic	3,770	3,553	3,770	3,576
KS+ (100 MB) failures	9		11	
KS+ (1 MB) failures	36		54	

Table 16: Memory each approach holds for the 29 workloads (the 23 corpus programs at their largest inputs, miniVite built with the pass, six LLM configurations), summed over the workloads: the peak and the time-averaged memory held over the run, in MB, mean of three runs. The demand a job must be given is its RSS (default overcommit, a cgroup limit) or the larger of its commit and its RSS (strict overcommit). Every approach starts from the same exact request, the job's peak of commit and RSS under glibc. Pond holds the whole request; the row below it is the part in local DRAM. Failures count KS+'s re-executions over all workloads and runs.

Workload	Glibc request	Pond local	Dynamic	KS+ (fails)	MemPrintResize	MPR + dynamic
2mm	38	38	38	41 (0)	39	39
atax	32	33	32	29 (19)	33	32
cc1	134	134	116	1,140 (0)	133	115
darknet	534	287	533	534 (0)	287	286
gap-bfs	76	76	60	82 (2)	76	62
gap-cc	76	76	61	82 (2)	76	60
gap-pr	76	76	57	82 (2)	76	58
gap-sssp	137	134	109	135 (2)	136	109
gemm	30	30	30	36 (0)	31	31
hpccg	106	107	106	116 (0)	106	106
jacobi-2d	28	28	28	33 (3)	29	29
kripke	279	279	279	310 (3)	279	279
lua	66	66	62	67 (0)	67	62
lulesh	154	154	154	168 (0)	155	155
minife	273	269	273	295 (2)	271	271
npb-cg	30	30	30	34 (0)	31	31
npb-is	72	72	71	79 (3)	72	72
npb-mg	60	61	60	62 (3)	61	61
perl	65	65	61	63 (1)	64	61
python	114	114	98	105 (3)	115	98
quicksilver	54	54	54	79 (0)	53	53
sqlite	79	80	66	64 (1)	80	63
miniVite (pass)	34	31	29	37 (2)	27	24
llamacpp-135m-n32	403	380	330	271 (0)	175	134
llamacpp-135m-n128	404	380	404	343 (0)	177	176
llamacpp-360m-n64	767	768	767	738 (4)	419	418
llama2c-15M	67	67	66	96 (0)	67	67
llama2c-42M	201	201	201	210 (1)	176	175
llama2c-110M	515	515	509	598 (0)	452	450

Table 17: Per workload: the time-averaged memory held (MB, mean of three runs) when the demand is the larger of commit and RSS. KS+ with the 1 MB floor; its failures (re-executions per run) in parentheses. MPR: `libmpr` with its own profile.

	Demand = RSS						Demand = max(commit, RSS)					
	Jobs/h		Resp. (min)		Fails	TB	Jobs/h		Resp. (min)		Fails	TB
Load	0.8	1.2	0.8	1.2	0.8	0.8	0.8	1.2	0.8	1.2	0.8	0.8
Exact curve (glibc build)	30.5	30.3	661	1,237	0	5.0	27.7	27.7	799	1,393	0	5.0
Request (glibc build)	21.6	21.6	1,827	2,437	0	6.0	21.5	21.5	1,761	2,378	0	6.0
Pond	21.2	21.0	1,951	2,535	0	7.0	21.2	21.3	1,811	2,441	0	6.0
Dynamic provisioning	21.2	20.6	1,136	1,881	831	6.0	20.2	20.3	1,202	1,868	727	6.0
Dynamic provisioning, look-back	21.1	20.8	1,137	1,716	1,118	6.0	20.5	20.7	1,133	1,716	922	5.0
KS+	21.5	20.0	1,500	2,333	6,276	7.0	20.1	20.5	1,708	2,219	7,246	7.0
Exact curve (pass build)	34.6	34.8	342	894	0	5.0	32.5	32.6	356	940	0	5.0
MemPrintResize	25.0	25.2	1,225	1,806	0	6.0	24.9	25.0	1,111	1,741	0	5.0
MemPrintResize + dynamic	23.8	24.0	718	1,432	675	5.0	23.9	23.2	742	1,291	619	5.0
MemPrintResize + dynamic, look-back	24.6	24.4	665	1,261	961	5.0	24.1	24.2	683	1,205	749	5.0
MemPrintResize + Pond	25.0	25.2	1,230	1,819	0	6.0	24.9	25.0	1,132	1,745	0	5.0
MemPrintResize, held-out profile	24.3	24.4	1,330	1,936	0	6.0	24.0	24.1	1,242	1,872	0	6.0
MemPrintResize, held-out, + dynamic	24.1	23.3	797	1,408	689	5.0	22.6	22.6	890	1,461	613	5.0

Table 18: Throughput (jobs per hour) and mean response time at offered loads 0.8 and 1.2, failures per stream at 0.8 (OOM kills, and dynamic provisioning's restarts), and the total memory (32 nodes plus the 1 TB pool) each policy needs at load 0.8 for a mean bounded slowdown within 1.5 times the glibc build's exact curve at 128 GB per node, from a sweep over 64 to 320 GB per node.

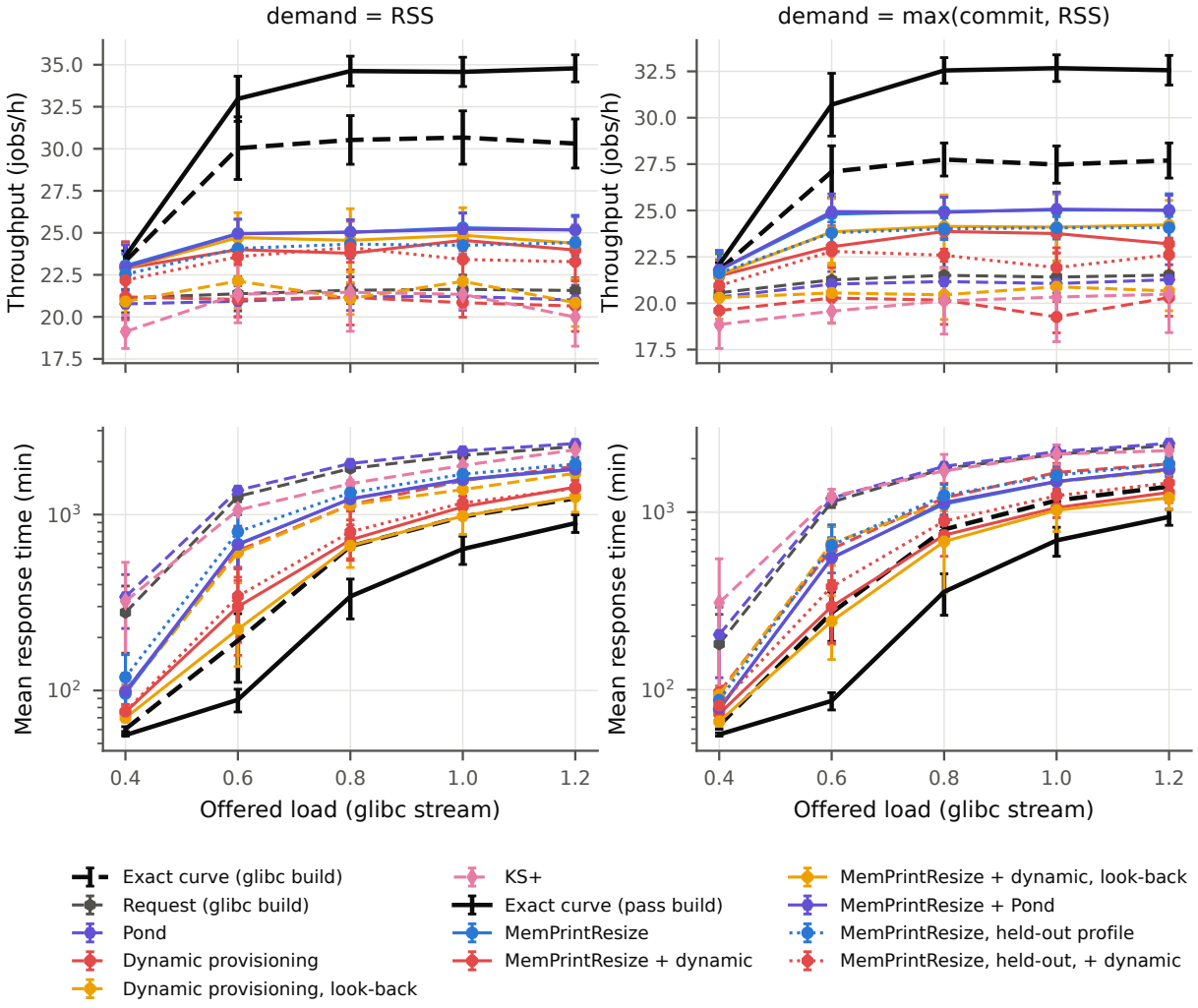


Figure 9: Throughput and mean response time against offered load in DMSched-Sim (32 nodes of 128 GB, a 1 TB pool, 3,000 jobs, mean and standard deviation over three seeds). Each job is one of the 29 workloads, built without the pass (dashed) or with it (solid), and requests what its program allocates. Left: the job must be given its RSS. Right: the larger of its commit and its RSS.

	Jobs/h				Resp. (min)			
Request over allocation	+0%	+25%	+60%	+100%	+0%	+25%	+60%	+100%
Exact curve (glibc build)	30.5	30.5	30.5	30.5	661	661	661	661
Request (glibc build)	21.6	14.5	9.4	6.8	1,827	3,822	6,840	10,552
Pond	21.2	12.8	8.3	6.0	1,951	4,677	8,414	12,566
Dynamic provisioning	21.2	21.2	19.0	15.4	1,136	1,198	1,520	2,164
Dynamic provisioning, look-back	21.1	19.9	16.8	13.7	1,137	1,488	2,058	2,934
KS+	21.5	19.5	19.1	18.9	1,500	2,060	2,085	2,079
Exact curve (pass build)	34.6	34.6	34.6	34.6	342	342	342	342
MemPrintResize	25.0	25.0	25.0	25.0	1,225	1,225	1,225	1,225
MemPrintResize + dynamic	23.8	23.8	23.8	23.8	718	718	718	718
MemPrintResize + dynamic, look-back	24.6	24.6	24.6	24.6	665	665	665	665
MemPrintResize + Pond	25.0	25.0	25.0	25.0	1,230	1,230	1,230	1,230
MemPrintResize, held-out profile	24.3	24.3	24.3	24.3	1,330	1,330	1,330	1,330
MemPrintResize, held-out, + dynamic	24.1	24.1	24.1	24.1	797	797	797	797

Table 19: Throughput and mean response time at offered load 0.8 (demand = RSS) when each job built without the pass requests more than its program allocates, as a user who over-requests would. The builds with the pass take their requests from the profile, so they do not change.

	Jobs/h			Resp. (min)			Lost	Actions	Profiling	
Policy	no cost	native	Pin	no cost	native	Pin	core-h	core-h	native	Pin
Exact curve (glibc build)	30.5	30.5	30.5	661	661	661	0	0	0	0
Request (glibc build)	21.6	21.6	21.6	1,827	1,827	1,827	0	0	0	0
Pond	21.2	21.2	21.2	1,951	1,951	1,951	0	0	0	0
Dynamic provisioning	21.2	20.6	20.6	1,136	1,271	1,271	3,699	395	0	0
Dynamic provisioning, look-back	21.1	21.5	–	1,137	1,142	–	2,773	1,098	0	–
KS+	21.5	21.5	21.5	1,500	1,500	1,500	8,322	0	0	0
Exact curve (pass build)	34.6	34.5	34.5	342	333	333	0	0	0	0
MemPrintResize	25.0	25.1	3.1	1,225	1,229	6,981	0	0	198	47,539
MemPrintResize + dynamic	23.8	24.0	3.1	718	803	7,766	2,908	396	198	47,539
MemPrintResize + dynamic, look-back	24.6	24.1	–	665	796	–	2,448	1,195	198	–
MemPrintResize + Pond	25.0	25.1	3.1	1,230	1,237	6,944	0	0	198	47,539
MemPrintResize, held-out profile	24.3	24.2	–	1,330	1,337	–	0	0	198	–
MemPrintResize, held-out, + dynamic	24.1	23.7	–	797	832	–	2,827	400	198	–

Table 20: Each policy's own costs at offered load 0.8, demand = RSS, mean of three seeds. Jobs/h and response time without the policies' costs, and with them, MemPrintResize profiling either at native speed or with the Pin tool. Lost: core-hours of work lost to OOM kills and restarts. Actions: core-hours that resizes and Pond's mitigations add to the runs. Profiling: core-hours of the profiling runs (one per workload, at its first job in the stream).

MemPrint curve	Median margin	90th percentile	Response (min)
Full trace, bytes, references	0.06	0.58	1,847
Full trace, bytes, seconds	0.08	0.44	1,476
Full trace, pages, references	0.03	0.56	1,942
Full trace, pages, seconds	0.05	0.41	1,423
W5, bytes, references	0.04	0.58	1,807
W5, bytes, seconds	0.04	0.56	1,820
W5, pages, references	0.04	0.58	1,979
W5, pages, seconds	0.04	0.49	1,705
Exact RSS peak	–	–	436
Exact RSS curve	–	–	91
User request	–	–	2,197

Table 21: MemPrint's footprint curves under the RSS truth, with the Pin tool's clock and unit options: the margin (headroom as a share of the curve's peak) each program needs for the curve never to fall below its RSS, over 23 programs, and the mean response time at 80% load with calibrated margins.

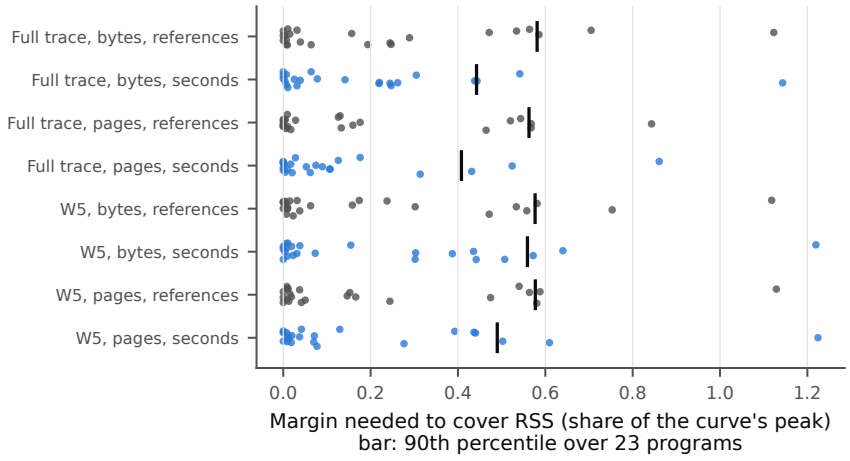


Figure 10: The margin each corpus program needs (dots; blue: wall-clock axis) and the 90th percentile over programs (bars), which sets the calibrated margin.